Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mfunc functions

Synopsis

The functions prefixed with Mfunc make up the Function Development module. The Function Development module allows programmers to define functions to extend Aurora Imaging Library's functionality. Using this toolkit, you can implement functions and integrate them directly into Aurora Imaging Library, where they behave like standard Aurora Imaging Library functions (for example, respecting error handling and tracing).

Functions

MfuncAlloc
MfuncAllocId
MfuncAllocScript
MfuncBufAncestorId
MfuncBufAncestorOffsetBand
MfuncBufAncestorOffsetBit
MfuncBufAncestorOffsetX
MfuncBufAncestorOffsetY
MfuncBufAttribute
MfuncBufFormat
MfuncBufHostAddress
MfuncBufHostAddressBand
MfuncBufId
MfuncBufMaxValue
MfuncBufMinValue
MfuncBufOwnerSystemId
MfuncBufOwnerSystemType
MfuncBufParentId
MfuncBufParentOffsetBand
MfuncBufParentOffsetX
MfuncBufParentOffsetY
MfuncBufPhysicalAddress
MfuncBufPhysicalAddressBand
MfuncBufPitch
MfuncBufPitchByte
MfuncBufSizeBand
MfuncBufSizeBit
MfuncBufSizeX
MfuncBufSizeY
MfuncBufType
MfuncCall
MfuncControl
MfuncErrorReport

MfuncFree
MfuncFreeId
MfuncInquire
MfuncParam
MfuncParamCheck
MfuncParamValue

MfuncAlloc

Synopsis

Allocate an Aurora Imaging Library function context for your user-defined function.

Syntax

```
AIL_ID MfuncAlloc(
    AIL_CONST_TEXT_PTR FunctionName,          //in
    AIL_INT ParameterNum,                     //in
    AIL_FUNC_FUNCTION_PTR CalleeFunctionPtr,  //in
    AIL_CONST_TEXT_PTR CalleeFunctionDLLName, //in
    AIL_CONST_TEXT_PTR CalleeFunctionName,    //in
    AIL_INT UserFunctionOpcode,               //in
    AIL_INT64 InitFlag,                       //in
    AIL_ID *CBasedContextFuncIdPtr            //out
)
```

Description

This function allows you to allocate an Aurora Imaging Library function context for the current user-defined function. Call **MfuncAlloc**() in the caller function of the user-defined function. **MfuncAlloc**() signals the creation of a user-defined Aurora Imaging Library function context, and should be the first Aurora Imaging Library function called in the caller function. Once the function context is allocated, your function is known as a user-defined Aurora Imaging Library function. A user-defined Aurora Imaging Library function is considered a standard Aurora Imaging Library function, respecting all Aurora Imaging Library environment controls, such as tracing and error handling.

When defining the function context, you must specify a unique opcode for the user-defined function. The opcode is used to locate the user-defined function.

Aurora Imaging Library user-defined functions can be grouped into user-defined modules or remain ungrouped. To specify whether a user-defined function is grouped or not, and if grouped, assign it to a specific user-defined module, pass an appropriate opcode to the UserFunctionOpcode parameter. Specify the opcode as a combination of two values: an offset from 0 to 63 and a label either identifying the module in which to group the function (M_USER_MODULE_n, where n is a number between 1 and 7) or identifying that the function is ungrouped (M_USER_FUNCTION). A user-defined module can have up to 64 functions, and there can be up to 64 ungrouped functions.

Parameters

FunctionName

Specifies the name of the user-defined function.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the name of the user-defined function																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

		c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FunctionName*") [1]	Specifies the name of the user-defined function. This is the same as the name of the current caller function. This is also the name that you will use to call the user-defined Aurora Imaging Library function in your application. The name of the function must be a null-terminated string.															

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ParameterNum

Specifies the number of parameters passed to the current user-defined function. Note that the number of parameters should not exceed 16.

CalleeFunctionPtr

Specifies the address of the callee function.

The callee function must be declared as follows:

```
void MFTYPE CalleeFunction(
    AIL_ID FunctionId
)
```

Parameters

FunctionId

Specifies the identifier of the user-defined Aurora Imaging Library function that references this callee function. This is the identifier returned by the **MfuncAlloc**() function.

If your callee function's code will be compiled into a library file, you can set this parameter to **M_NULL**.

CalleeFunctionDLLName

Specifies the name of the library file which contains the callee function's code. The library file must be visible to the system executing the callee function.

When developing a user-defined Aurora Imaging Library function which will be used in a Distributed Aurora Imaging Library application, it is recommended that a copy of the library file be placed on all the remote systems in the cluster, in a directory which is part of the system's path environment variable, using MappFileOperation() with M_FILE_COPY_AIL_USER_DLL.

> If the library file is not located in a directory which is part of the path environment variable, you can alternatively provide the full path to the file.

If your callee function is included in your application, you can set this parameter to **M_NULL**.

CalleeFunctionName

Specifies the name of the callee function to export from the library file.

This parameter must be set if a library file is provided to the CalleeFunctionDLLName parameter.

If your callee function is included in your application, you can set this parameter to **M_NULL**.

UserFunctionOpcode

Specifies the opcode to use when an application executes the user-defined Aurora Imaging Library function. The opcode is used to locate the user-defined function and identifies which user-defined module, if any, the function belongs to.

To specify a label identifying that the function is ungrouped, or a label identifying the module in which the function is grouped, specify one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the label identifying that the function is grouped or ungrouped																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_USER_FUNCTION +	Specifies that the user-defined function is an ungrouped user-defined function. You must specify a combination value from the following table: • To specify the offset	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_MODULE_*n* +	Specifies the module in which to group the user-defined function, where *n* is a value between 1 and 7, inclusive. You must specify a combination value from the following table: • To specify the offset	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for any of the possible values of the UserFunctionOpcode parameter.

You must add the following value to the above-mentioned values to specify the offset.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the offset																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
0 <= *Value* <= 63	Specifies the offset of the user-defined function among the ungrouped functions or among the functions in a specified group. When allocating multiple user-defined functions in a single user-defined module, you must set a different offset for each new function in the module. For example, M_USER_MODULE_3 + 12. To allocate a user-defined function that is not grouped in a user-defined module, use M_USER_FUNCTION + the offset; you must set a different offset for each new ungrouped function.	c		e	f	g	a		i		j	k	l	m	n	b

InitFlag

Specifies more information about the user-defined Aurora Imaging Library function.

The values below allow you to set whether the callee function is executed asynchronously or synchronously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether the callee function is executed asynchronously or synchronously																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. If the system on which the function context is allocated has a remote processor, the default is the same as M_ASYNCHRONOUS_FUNCTION + M_REMOTE, if the system does not have a remote processor, the default is the same as M_SYNCHRONOUS_FUNCTION + M_LOCAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ASYNCHRONOUS_FUNCTION +	Specifies that the user-defined function will not wait for the callee function to finish executing before executing the next function (typically MfuncFree()) in the caller function. You must specify a combination value from the following table: • To set whether the callee function can be executed remotely	c		e	f	g	a		i		j	k	l	m	n	b
M_SYNCHRONOUS_FUNCTION +	Specifies that the user-defined function will wait for the callee function to finish executing before executing the next function in the caller function. You must specify a combination value from the following table: • To set whether the callee function can be executed remotely	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_ASYNCHRONOUS_FUNCTION; M_SYNCHRONOUS_FUNCTION.

You must add one of the following values to the above-mentioned values to set whether the callee function can be executed remotely.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether the callee function can be executed remotely																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_LOCAL	Specifies that the callee function must be executed by the Host processor. Note that Aurora Imaging Library functions called from the callee function will still be executed on their target system. This value cannot be added to M_ASYNCHRONOUS_FUNCTION.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMOTE	Specifies that the callee function will be executed remotely, if possible. If you select M_REMOTE, but do not compile your callee function with a cross-compiler specific to the target processor, an error will be generated. For more information, see the Steps to create a user-defined Aurora Imaging Library function section of Chapter 68: The Aurora Imaging Library function development module.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_SYNCHRONOUS_FUNCTION.

You can add one of the following values to the above-mentioned value to set the type of the user-defined Aurora Imaging Library function.

Note that both M_ALLOC and M_FREE imply that the user-defined function is M_SYNCHRONOUS_FUNCTION, and cannot be added to M_ASYNCHRONOUS_FUNCTION.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting the type of user-defined Aurora Imaging Library function																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALLOC	Specifies that the user-defined function is an allocation function, used to allocate a user-defined Aurora Imaging Library object on a required system.	c		e	f	g	a		i		j	k	l	m	n	b
M_FREE	Specifies that the user-defined function frees an object allocated using a user-defined Aurora Imaging Library allocation function. Note that when this value is used, the user-defined Aurora Imaging Library function must accept the Aurora Imaging Library identifier of the object to free as its only parameter.	c		e	f	g	a		i		j	k	l	m	n	b

CBasedContextFuncIdPtr

Specifies the address of the variable in which to store the Aurora Imaging Library identifier provided for the user-defined function. Since the **MfuncAlloc**() function also returns the Aurora Imaging Library identifier, you can set this parameter to **M_NULL**.

Return value

The returned value is the function context identifier if the allocation is successful. If allocation fails, **M_NULL** is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncAllocId

Synopsis

Associate an Aurora Imaging Library identifier with a user-defined object.

Syntax

```
AIL_ID MfuncAllocId(
    AIL_ID ContextFuncId,    //in
    AIL_INT64 AilObjectType, //in
    void *UserObjectPtr      //in
)
```

Description

This function allows you to associate an Aurora Imaging Library identifier with a user-defined object (such as, a structure or an array). You can associate an Aurora Imaging Library identifier with a user object so that it is recognized by Aurora Imaging Library and treated as a standard Aurora Imaging Library object during tracing or error handling. Once the object is associated with an identifier, the object is known as a user-defined Aurora Imaging Library object.

The parameters passed to **MfuncAllocId**() do not allow you to specify whether the user-defined object is allocated on the Host or on a system with an on-board processor. For information on creating user-defined objects on systems with on-board processors and associating these object with Aurora Imaging Library identifiers, see the Associating an Aurora Imaging Library identifier with a user-defined object section of Chapter 68: The Aurora Imaging Library function development module.

You must also specify an Aurora Imaging Library type for your object (AilObjectType parameter). Aurora Imaging Library allows up to 32 different user-defined Aurora Imaging Library object types.

Use the Aurora Imaging Library identifier to refer to the user-defined Aurora Imaging Library object. To refer to the actual data grouped into this user-defined object, use the pointer to the object (UserObjectPtr parameter).

To inquire about the type of this object, or the pointer to it, use the MfuncInquire() function.

Parameters

ContextFuncId

Specifies an Aurora Imaging Library function identifier. This parameter can be set to the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an Aurora Imaging Library function identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default Aurora Imaging Library function identifier. Use this value to create user-defined Aurora Imaging Library objects outside of a user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b

Function identifier	Specifies the identifier of a user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b

AilObjectType

Specifies the Aurora Imaging Library type of the user-defined object to be associated with an Aurora Imaging Library identifier. To specify the type, you must specify one of the groups listed in the table below and combine it with an offset. The two object groups allow you to distinguish between custom created objects, especially when the created objects are to be used in different Aurora Imaging Library modules.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Aurora Imaging Library type of user-defined object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_USER_OBJECT_1 +	Specifies that the user object falls under the first group of user object types. You must specify a combination value from the following table: • To set the offset	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_OBJECT_2 +	Specifies that the user object falls under the second group of user object types. You must specify a combination value from the following table: • To set the offset	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for any of the possible values of the AilObjectType parameter.

You must add the following value to the above-mentioned values to set the offset.

The offset allows you to distinguish between the different object types of the same group (for example, M_USER_OBJECT_1 + 0x0001).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For distinguishing between the different object types																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value	Specifies the offset within the selected object type group. The value must have only one of its 16 least significant bits set.	c		e	f	g	a		i		j	k	l	m	n	b

UserObjectPtr

Specifies the address of the user-defined object that is to be associated with an Aurora Imaging Library identifier. Note that user-defined objects stored in the memory of an on-board processor are not accessible from Host, and vice versa.

This object can be a structure, an array, or any other data type.

Return value

The returned value is the Aurora Imaging Library identifier associated with the object if the allocation is successful. If allocation fails, **M_NULL** is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncAllocScript

Synopsis

Allocate an Aurora Imaging Library function context for your script-based user-defined Aurora Imaging Library function.

Syntax

```
AIL_ID MfuncAllocScript(
    AIL_CONST_TEXT_PTR FunctionName,        //in
    AIL_INT ParameterNum,                   //in
    AIL_CONST_TEXT_PTR InterpreterLanguage, //in
    AIL_CONST_TEXT_PTR ScriptFileName,      //in
    AIL_CONST_TEXT_PTR ScriptFunctionName,  //in
    AIL_INT ScriptFunctionOpcode,           //in
    AIL_INT64 InitFlag,                     //in
    AIL_ID *ScriptBasedContextFuncIdPtr     //out
)
```

Description

This function allows you to allocate an Aurora Imaging Library function context for the current script-based user-defined Aurora Imaging Library function. Being script-based, the function will, instead of calling a C function, run a function in a script. Call **MfuncAllocScript**() in the caller function of the user-defined function. **MfuncAllocScript**() signals the creation of a user-defined Aurora Imaging Library function, and should be the first Aurora Imaging Library function called in the caller function. It will also allocate an Aurora Imaging Library function context identifier for the current function. You should use this Aurora Imaging Library function context identifier as an argument for other Mfunc...() functions used during the user-defined function's execution. Once the function context is allocated, your function is known as a script-based user-defined Aurora Imaging Library function. A script-based user-defined Aurora Imaging Library function is considered a standard Aurora Imaging Library function, respecting all Aurora Imaging Library environment controls, such as tracing and error handling. The script that you provided as the callee-portion of the function is compiled/interpreted at runtime.

When defining the function context, you must specify a unique opcode for the script-based user-defined function. The opcode is used to locate the script-based user-defined function when an application calls the script-based user-defined function. You must also specify the language in which the script is written.

Script-based user-defined Aurora Imaging Library functions can be grouped into user-defined script modules or remain ungrouped. To specify whether a user-defined function is grouped or not, and if grouped, assign it to a specific user-defined script module, pass an appropriate opcode to the ScriptFunctionOpcode parameter. Specify the opcode as a combination of two values: an offset from 0 to 63 and a label either identifying the module in which to group the function (M_SCRIPT_MODULE...) or identifying that the function is ungrouped (M_SCRIPT_FUNCTION). A user-defined script module can have up to 64 functions, and there can be up to 64 ungrouped functions.

Parameters

FunctionName

Specifies the name of the script-based user-defined Aurora Imaging Library function.

For specifying the name of the user-defined function																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*FunctionName*") 1	Specifies the name of the script-based user-defined Aurora Imaging Library function. This is the same as the name of the current caller function. This is also the name that you will use to call the user-defined function in your application. The name of the function must be a null-terminated string.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ParameterNum

Specifies the number of parameters to be registered for the current user-defined function.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of parameters																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
0 <= *Value* <= 15	Specifies the number of parameters that you register for the function using MfuncParam() (or one of the type-specific versions of this function such as MfuncParamAILInt()). This should correspond to the number of parameters passed to the current user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b

InterpreterLanguage

Specifies the language and language version in which the script is written.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the path to the language interpreter.																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERPRETER_C_PYTHON3X	Specifies that the first successfully loaded 3.X version of CPython is used. Aurora Imaging Library will attempt to load the most recent 3.X version of CPython among the supported versions. > Note that the script must be written using this version of the language.	c		e	f	g	a		i		j	k	l	m	n	b

M_INTERPRETER_C_PYTHON36	[This is only applicable to Linux] Specifies that a CPython 3.6 based script is used. This is only supported if CPython 3.6 is the default Python version for the Linux distribution where Aurora Imaging Library is installed. > Note that the script must be written using this version of the language.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPRETER_C_PYTHON37	Specifies that a CPython 3.7 based script is used. > Note that the script must be written using this version of the language.	c		e	f	g	a		i		j	k	l	m	n	b
	Operating system specific															
	[This is only applicable to Linux] For Aurora Imaging Library under Linux, this version is only supported if CPython 3.7 is the default Python version for the Linux distribution where Aurora Imaging Library is installed.															
M_INTERPRETER_C_PYTHON38	[This is only applicable to Linux] Specifies that a CPython 3.8 based script is used. This is only supported if CPython 3.8 is the default Python version for the Linux distribution where Aurora Imaging Library is installed. > Note that the script must be written using this version of the language.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPRETER_C_PYTHON39	Specifies that a CPython 3.9 based script is used. > Note that the script must be written using this version of the language.	c		e	f	g	a		i		j	k	l	m	n	b
	Operating system specific															
	[This is only applicable to Linux] For Aurora Imaging Library under Linux, this version is only supported if CPython 3.9 is the default Python version for the Linux distribution where Aurora Imaging Library is installed.															

M_INTERPRETER_C_PYTHON310	[This is only applicable to Linux] Specifies that a CPython 3.10 based script is used. This is only supported if CPython 3.10 is the default Python version for the Linux distribution where Aurora Imaging Library is installed. > Note that the script must be written using this version of the language.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPRETER_C_PYTHON311	Specifies that a CPython 3.11 based script is used. > Note that the script must be written using this version of the language.	c		e	f	g	a		i		j	k	l	m	n	b
	Operating system specific															
	[This is only applicable to Linux] For Aurora Imaging Library under Linux, this version is only supported if CPython 3.11 is the default Python version for the Linux distribution where Aurora Imaging Library is installed.															
M_INTERPRETER_CSHARP	Specifies that C# based code is used. > Note that .NET Core is not supported by the Aurora Imaging Library Interpreter in a .NET program.	c		e	f	g	a		i		j	k	l	m	n	b

ScriptFileName

Specifies the name and path of the file containing the user-defined script. The script file must be accessible to the computer executing the script.

> If the script file is not located in the current directory, the full path to the file is required.

When developing a script-based user-defined Aurora Imaging Library function that will be used in a Distributed Aurora Imaging Library application, you must specify the full path to the location of the scripts on the remote computer. Ensure that all included scripts are located together on the remote computer.

ScriptFunctionName

Specifies the name of the function to export from the script file.

> Note that an error is generated if no function name is provided.

ScriptFunctionOpcode

Specifies the opcode to use to locate the script-based user-defined Aurora Imaging Library function when an application calls the function. The opcode also serves to identify the module in which to group the function, if any.

To set a label identifying that the script-based user-defined Aurora Imaging Library function is ungrouped, or

a label identifying the module in which it is grouped, set one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting the label identifying that the function is grouped or ungrouped																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SCRIPT_FUNCTION +	Specifies that the script-based user-defined Aurora Imaging Library function is an ungrouped user-defined Aurora Imaging Library function. You must specify a combination value from the following table: • To set the offset	c		e	f	g	a		i		j	k	l	m	n	b
M_SCRIPT_MODULE_1 +	Specifies that the script-based user-defined Aurora Imaging Library function will be grouped in the first module. You must specify a combination value from the following table: • To set the offset	c		e	f	g	a		i		j	k	l	m	n	b
M_SCRIPT_MODULE_2 +	Specifies that the script-based user-defined Aurora Imaging Library function will be grouped in the second module. You must specify a combination value from the following table: • To set the offset	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for any of the possible values of the ScriptFunctionOpcode parameter.

You must add the following value to the above-mentioned values to set the offset.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting the offset																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

0 <= *Value* <= 63	Sets the offset of the script-based user-defined Aurora Imaging Library function among the ungrouped user-defined Aurora Imaging Library functions or among those in the specified group. When allocating multiple user-defined Aurora Imaging Library functions in a single user-defined module, you must set a different offset for each new user-defined Aurora Imaging Library function in the module (for example, M_SCRIPT_MODULE_1 + 12). To allocate a script-based user-defined Aurora Imaging Library function that is not grouped in a user-defined module, use M_SCRIPT_FUNCTION + the offset; you must set a different offset for each new ungrouped user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b

InitFlag

Specifies more information about the script-based user-defined Aurora Imaging Library function.

The values below allow you to set whether the script-based user-defined Aurora Imaging Library function is executed asynchronously or synchronously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether the script-based user-defined Aurora Imaging Library function is executed asynchronously or synchronously																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. If the system on which the function context is allocated has a remote processor, the default is the same as M_ASYNCHRONOUS_FUNCTION + M_REMOTE, if the system does not have a remote processor, the default is the same as M_SYNCHRONOUS_FUNCTION + M_LOCAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ASYNCHRONOUS_FUNCTION +	Specifies that the script-based user-defined Aurora Imaging Library function will not wait for the callee script/function to finish executing before executing the next function (typically MfuncFree()) in the caller function. You must specify a combination value from the following table: • To set whether the script-based user-defined Aurora Imaging Library function can be executed remotely	c		e	f	g	a		i		j	k	l	m	n	b

M_SYNCHRONOUS_FUNCTION +	Specifies that the script-based user-defined Aurora Imaging Library function will wait for the callee script/function to finish executing before executing the next function in the caller function. You must specify a combination value from the following table: • To set whether the script-based user-defined Aurora Imaging Library function can be executed remotely	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_ASYNCHRONOUS_FUNCTION; M_SYNCHRONOUS_FUNCTION.

You must add one of the following values to the above-mentioned values to set whether the script-based user-defined Aurora Imaging Library function can be executed remotely.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether the script-based user-defined Aurora Imaging Library function can be executed remotely																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LOCAL	Specifies that the script-based user-defined Aurora Imaging Library function must be executed by the Host processor. Note that Aurora Imaging Library functions called from the script-based user-defined Aurora Imaging Library function will still be executed on their target system. This value cannot be added to M_ASYNCHRONOUS_FUNCTION.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMOTE	Specifies that the script-based user-defined Aurora Imaging Library function will be executed remotely, if possible. For more information, see the Steps to create a user-defined Aurora Imaging Library function section of Chapter 68: The Aurora Imaging Library function development module.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_SYNCHRONOUS_FUNCTION.

You can add one of the following values to the above-mentioned value to set the type of the script-based user-defined Aurora Imaging Library function.

Note that both M_ALLOC and M_FREE imply that the script-based user-defined Aurora Imaging Library function is M_SYNCHRONOUS_FUNCTION, and cannot be added to M_ASYNCHRONOUS_FUNCTION.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting the type of script-based user-defined Aurora Imaging Library function																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_ALLOC	Specifies that the script-based user-defined Aurora Imaging Library function is an allocation function, used to allocate a user-defined Aurora Imaging Library object on a required system.	c		e	f	g	a		i		j	k	l	m	n	b
M_FREE	Specifies that the script-based user-defined Aurora Imaging Library function frees an object allocated using a user-defined Aurora Imaging Library allocation function. Note that when this value is used, the user-defined Aurora Imaging Library function must accept the Aurora Imaging Library identifier of the object to free as its only parameter.	c		e	f	g	a		i		j	k	l	m	n	b

ScriptBasedContextFuncIdPtr

Specifies the address of the variable in which to store the Aurora Imaging Library identifier of the allocated Aurora Imaging Library function context. Since the **MfuncAllocScript**() function also returns the Aurora Imaging Library identifier, you can set this parameter to **M_NULL**.

Return value

The returned value is the Aurora Imaging Library function context identifier if the allocation is successful. If allocation fails, **M_NULL** is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufAncestorId

Synopsis

Return the Aurora Imaging Library identifier of the ancestor buffer.

Syntax

```
AIL_ID MfuncBufAncestorId(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the Aurora Imaging Library identifier of the ancestor buffer.

Only child buffers have an ancestor buffer. The ancestor buffer is the buffer from which the specified buffer (BufferInfoHandle) ultimately originated. It is the root buffer; it does not have a parent buffer (it is not a child buffer of another buffer).

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the Aurora Imaging Library identifier of the specified Aurora Imaging Library buffer's ancestor buffer.

Remark

- This function is equivalent to using MbufInquire() with M_ANCESTOR_ID, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufAncestorOffsetBand

Synopsis

Return the band offset relative to the ancestor buffer.

Syntax

```
AIL_INT MfuncBufAncestorOffsetBand(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the band offset of the specified buffer relative to its ancestor buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the band offset of the specified Aurora Imaging Library buffer relative to its ancestor buffer.

Remark

- This function is equivalent to using MbufInquire() with M_ANCESTOR_OFFSET_BAND, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufAncestorOffsetBit

Synopsis

Return the bit offset relative to the ancestor buffer.

Syntax

```
AIL_INT MfuncBufAncestorOffsetBit(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the bit offset of the specified Aurora Imaging Library buffer relative its ancestor buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the bit offset of the specified Aurora Imaging Library buffer relative its ancestor buffer.

Remark

- This function is equivalent to using MbufInquire() with M_ANCESTOR_OFFSET_BIT, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufAncestorOffsetX

Synopsis

Return the X-offset relative to the ancestor buffer.

Syntax

```
AIL_INT MfuncBufAncestorOffsetX(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the X-offset of the specified Aurora Imaging Library buffer relative its ancestor buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the X-offset of the specified Aurora Imaging Library buffer relative its ancestor buffer.

Remark

- This function is equivalent to using MbufInquire() with M_ANCESTOR_OFFSET_X, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufAncestorOffsetY

Synopsis

Return the Y-offset relative to the ancestor buffer.

Syntax

```
AIL_INT MfuncBufAncestorOffsetY(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the Y-offset of the specified Aurora Imaging Library buffer relative its ancestor buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the Y-offset of the specified Aurora Imaging Library buffer relative its ancestor buffer.

Remark

- This function is equivalent to using MbufInquire() with M_ANCESTOR_OFFSET_Y, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufAttribute

Synopsis

Return the requested format of the specified buffer as set using MbufAlloc...() with Attribute.

Syntax

```
AIL_INT64 MfuncBufAttribute(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the requested format of the specified buffer as set using MbufAlloc...() with Attribute.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is a bit-encoded value that specifies the buffer's attributes, set at allocation.

Remark

- This function is equivalent to using MbufInquire() with M_EXTENDED_ATTRIBUTE, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufFormat

Synopsis

Return the actual format of the specified buffer.

Syntax

```
AIL_INT64 MfuncBufFormat(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the requested format of the specified buffer as set using MbufAlloc...() with Attribute.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is a bit-encoded value that specifies the buffer's current formatting.

Remark

- This function is equivalent to using MbufInquire() with M_EXTENDED_FORMAT, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufHostAddress

Synopsis

Returns the Host address of the buffer.

Syntax

```
void *MfuncBufHostAddress(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the Host address of the specified Aurora Imaging Library buffer, if the buffer is visible from the Host address space and is not a planar 3-band buffer.

For a planar 3-band buffer, you can determine its Host address using MfuncBufHostAddressBand().

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the Host address of the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_HOST_ADDRESS, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufHostAddressBand

Synopsis

Returns the Host address of a band of the buffer.

Syntax

```
void *MfuncBufHostAddressBand(
    AIL_BUFFER_INFO BufferInfoHandle, //in
    AIL_INT Band                      //in
)
```

Description

This function returns the Host address of a band of the specified Aurora Imaging Library buffer, if the buffer is a planar 3-band buffer.

Parameters

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Band

Specifies which band's address to return.

Return value

The returned value is the Host address of a band of the specified Aurora Imaging Library buffer.

Remark

- This function provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufId

Synopsis

Return the Aurora Imaging Library identifier of the buffer.

Syntax

```
AIL_ID MfuncBufId(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the Aurora Imaging Library identifier of the specified buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the Aurora Imaging Library identifier of the specified Aurora Imaging Library buffer.

Remark

- This function provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufMaxValue

Synopsis

Return the maximum pixel value possible in the buffer.

Syntax

```
AIL_DOUBLE MfuncBufMaxValue(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the maximum pixel value possible in the specified Aurora Imaging Library buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the maximum pixel value possible in the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_MAX, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufMinValue

Synopsis

Return the minimum pixel value possible in the buffer.

Syntax

```
AIL_DOUBLE MfuncBufMinValue(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the minimum pixel value possible in the specified buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the minimum pixel value possible in the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_MIN, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufOwnerSystemId

Synopsis

Return the Aurora Imaging Library identifier of the system on which the buffer has been allocated.

Syntax

```
AIL_ID MfuncBufOwnerSystemId(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the Aurora Imaging Library identifier of the system on which the specified Aurora Imaging Library buffer has been allocated.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the Aurora Imaging Library identifier of the system on which the specified Aurora Imaging Library buffer has been allocated.

Remark

- This function is equivalent to using MbufInquire() with M_OWNER_SYSTEM, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufOwnerSystemType

Synopsis

Return the type of system on which the buffer has been allocated.

Syntax

```
AIL_INT MfuncBufOwnerSystemType(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the type of system on which the specified Aurora Imaging Library buffer has been allocated.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the type of system on which the specified Aurora Imaging Library buffer has been allocated.

Remark

- This function is equivalent to using MbufInquire() with M_OWNER_SYSTEM_TYPE, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufParentId

Synopsis

Return the Aurora Imaging Library identifier of the parent buffer.

Syntax

```
AIL_ID MfuncBufParentId(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the Aurora Imaging Library identifier of the specified Aurora Imaging Library buffers' parent buffer.

Only child buffers have a parent buffer. The parent buffer is the buffer from which the specified buffer was defined. The parent buffer can itself have a parent buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the Aurora Imaging Library identifier of the specified Aurora Imaging Library buffers' parent buffer.

Remark

- This function is equivalent to using MbufInquire() with M_PARENT_ID, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufParentOffsetBand

Synopsis

Return the band offset relative to the parent buffer.

Syntax

```
AIL_INT MfuncBufParentOffsetBand(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the band offset of the specified buffer relative to its parent buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the band offset of the specified Aurora Imaging Library buffer relative to its parent buffer.

Remark

- This function is equivalent to using MbufInquire() with M_PARENT_OFFSET_BAND, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufParentOffsetX

Synopsis

Return the X-offset relative to the parent buffer.

Syntax

```
AIL_INT MfuncBufParentOffsetX(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the X-offset of the specified Aurora Imaging Library buffer relative its parent buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the X-offset of the specified Aurora Imaging Library buffer relative its parent buffer.

Remark

- This function is equivalent to using MbufInquire() with M_PARENT_OFFSET_X, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufParentOffsetY

Synopsis

Return the Y-offset relative to the parent buffer.

Syntax

```
AIL_INT MfuncBufParentOffsetY(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function returns the Y-offset of the specified Aurora Imaging Library buffer relative its parent buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the Y-offset of the specified Aurora Imaging Library buffer relative its ancestor buffer.

Remark

- This function is equivalent to using MbufInquire() with M_PARENT_OFFSET_Y, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufPhysicalAddress

Synopsis

Return the physical address of the buffer.

Syntax

```
AIL_UINT64 MfuncBufPhysicalAddress(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the physical address of the specified Aurora Imaging Library buffer, if it is not a planar 3-band buffer.

For a planar 3-band buffer, you can determine its physical address using MfuncBufPhysicalAddressBand().

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the physical address of the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_PHYSICAL_ADDRESS, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufPhysicalAddressBand

Synopsis

Returns the physical address of a band of the buffer.

Syntax

```
AIL_UINT64 MfuncBufPhysicalAddressBand(
    AIL_BUFFER_INFO BufferInfoHandle, //in
    AIL_INT Band                      //in
)
```

Description

This function returns the physical address of a band of the specified Aurora Imaging Library buffer, if the buffer is a planar 3-band buffer.

Parameters

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Band

Specifies which band's address to return.

Return value

The returned value is the physical address of a band of the specified Aurora Imaging Library buffer.

Remark

- This function provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufPitch

Synopsis

Return the number of pixels between the beginnings of any two adjacent lines of the buffer.

Syntax

```
AIL_INT MfuncBufPitch(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the number of pixels between the beginnings of any two adjacent lines of the specified Aurora Imaging Library buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the number of pixels between the beginnings of any two adjacent lines of the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_PITCH, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufPitchByte

Synopsis

Return the number of bytes between the beginnings of any two adjacent lines of the buffer.

Syntax

```
AIL_INT MfuncBufPitchByte(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the number of bytes between the beginnings of any two adjacent lines of the specified Aurora Imaging Library buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the number of bytes between the beginnings of any two adjacent lines of the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_PITCH_BYTE, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufSizeBand

Synopsis

Return the number of buffer color bands.

Syntax

```
AIL_INT MfuncBufSizeBand(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the number of color bands in the specified Aurora Imaging Library buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the number of color bands in the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_SIZE_BAND, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufSizeBit

Synopsis

Return the depth per band, in bits.

Syntax

```
AIL_INT MfuncBufSizeBit(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the depth per band, in bits, of the specified Aurora Imaging Library buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the depth per band, in bits, of the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_SIZE_BIT, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufSizeX

Synopsis

Return the width of the buffer, in pixels.

Syntax

```
AIL_INT MfuncBufSizeX(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the width, in pixels, of the specified Aurora Imaging Library buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the width, in pixels, of the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_SIZE_X, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufSizeY

Synopsis

Return the height of the buffer, in pixels.

Syntax

```
AIL_INT MfuncBufSizeY(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the height, in pixels, of the specified Aurora Imaging Library buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the height, in pixels, of the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_SIZE_Y, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncBufType

Synopsis

Return the buffer data type and depth.

Syntax

```
AIL_INT MfuncBufType(
    AIL_BUFFER_INFO BufferInfoHandle //in
)
```

Description

This function retrieves the data type and depth, in bits, of the specified Aurora Imaging Library buffer.

Parameter

BufferInfoHandle

Specifies the handle of the Aurora Imaging Library buffer. The buffer handle must be obtained using MfuncInquire() with M_BUFFER_INFO.

Return value

The returned value is the buffer data type and depth, in bits, of the specified Aurora Imaging Library buffer.

Remark

- This function is equivalent to using MbufInquire() with M_TYPE, but provides no parameter checking or error reporting.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncCall

Synopsis

Execute the callee function.

Syntax

```
AIL_INT MfuncCall(
    AIL_ID ContextFuncId //in
)
```

Description

This function executes the callee function of the user-defined Aurora Imaging Library function on the target system. The target system is determined by the values of the user-defined Aurora Imaging Library function parameters, registered in the caller function. (For more information, see the Caller/callee dynamics on a remote system section of Chapter 68: The Aurora Imaging Library function development module.) Call **MfuncCall**() from the caller function of the user-defined Aurora Imaging Library function. Note that you must call **MfuncCall**() from the same thread as MfuncAlloc() and MfuncFree(). If tracing and error reporting are enabled, the tracing and error messages will be reported to screen. You can control the error handling and tracing behavior as you would with other Aurora Imaging Library functions, using MappControl().

Note that if an AIL_ID parameter was registered in the caller function with MfuncParamAilId(), the validity of that identifier will be checked during the execution of **MfuncCall**(). If the identifier is not valid, the callee function is not executed.

Parameter

ContextFuncId

Specifies the identifier of the user-defined Aurora Imaging Library function.

The callee function executed using **MfuncCall**() must be declared as follows:

```
void MFTYPE CalleeFunction(
    AIL_ID ContextFuncId
)
```

Parameters

ContextFuncId

Specifies the identifier of the user-defined Aurora Imaging Library function that references this callee function. This is the identifier returned by the MfuncAlloc() function.

Return value

The returned value is **M_NULL** if an error occurred; otherwise, not null.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncControl

Synopsis

Control a user-defined Aurora Imaging Library function context.

Syntax

```
void MfuncControl(
    AIL_ID ContextFuncId,   //out
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function allows you to control the specified user-defined Aurora Imaging Library function context.

> Note that presented above is an example prototype of this function. You can call MfuncControl() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use MfuncControl() and not, for example, MfuncControlInt64(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

ContextFuncId

Specifies the identifier of the user-defined Aurora Imaging Library function context. Depending on the control type, the context must have been allocated using a specific Aurora Imaging Library allocation function.

ControlType

Specifies the user-defined Aurora Imaging Library function context setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ControlType** and **ControlValue** parameters.

- For script-based user-defined Aurora Imaging Library functions

The following ControlType and ControlValue parameter settings can be specified when the function context being controlled is a script-based user-defined Aurora Imaging Library function context allocated using MfuncAllocScript().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For script-based user-defined Aurora Imaging Library functions

ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_ADD_SCRIPT_REFERENCE		Adds the specified DLL to the list of DLLs that are referred to during runtime. This control type is not relevant for Python. INQ															
	AIL_TEXT("*String*") 1	Specifies the path and file name of the DLL your script/function requires to run. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*String*")).	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPILE		Sets when the script is reloaded from disk and recompiled. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_MODIFIED	Specifies that the script is recompiled only if the file on disk has changed. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ONCE	Specifies that the script is only compiled once.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEBUG_INFORMATION		Sets if the interpreter interface DLL must generate debug information for the script. This control type is not relevant for Python. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO	Specifies that the script is compiled in release mode, using optimization flags. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_YES	Specifies that the script is compiled in debug mode, and that the necessary debug information is generated. When choosing this control value, you must specifiy the directory in which to save the debug information using the M_DEBUG_INFORMATION_PATH control type.	c		e	f	g	a		i		j	k	l	m	n	b

M_DEBUG_INFORMATION_PATH		Sets the path of the directory in which the compiled version of the script and other files are created, if they are relevant for the scripting language. This control type is not relevant for Python. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*String*") 1	Specifies the path of the directory in which the debug information will be stored. This is also the directory in which the compiled version of the script is created. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*String*")). > The compiled version of the script (assembly file) will only be created in the specified directory if you choose to generate debug information using M_DEBUG_INFORMATION.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Type-specific versions of the function when using a C compiler

```
void MfuncControlInt64 (AIL_ID ContextFuncId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MfuncControlDouble (AIL_ID ContextFuncId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncErrorReport

Synopsis

Report an error message.

Syntax

```
AIL_INT MfuncErrorReport(
    AIL_ID ContextFuncId,                //in
    AIL_INT ErrorCode,                   //in
    AIL_CONST_TEXT_PTR ErrorMessage,     //in
    AIL_CONST_TEXT_PTR ErrorSubMessage1, //in
    AIL_CONST_TEXT_PTR ErrorSubMessage2, //in
    AIL_CONST_TEXT_PTR ErrorSubMessage3  //in
)
```

Description

This function allows you to log an error message in a user-defined Aurora Imaging Library function using the Aurora Imaging Library error handling mechanism. You can call the **MfuncErrorReport**() function from both the callee and the caller functions of your user-defined Aurora Imaging Library function, but only after a call to MfuncAlloc() and before a call to MfuncFree(). When **MfuncErrorReport**() is called, Aurora Imaging Library will treat your error as a normal Aurora Imaging Library error. To report the logged messages to screen, you must enable error reporting using the MappControl() function. These errors can also be read using the standard Aurora Imaging Library error functions (MappGetError()), which is especially useful when you want to keep error reporting disabled.

If you report an error with an error code set to M_NULL, you will reset any pending internal error that an Aurora Imaging Library function call might have generated inside the user-defined Aurora Imaging Library function. You must reset the pending error if you don't want the Aurora Imaging Library error message to be reported. If you don't reset the pending error at the end of the callee function, and you don't report your own error, Aurora Imaging Library will detect any pending error and report the error message, prefixed with the name of the user-defined Aurora Imaging Library function.

Parameters

ContextFuncId

Specifies the Aurora Imaging Library function identifier. This parameter can be set to the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the Aurora Imaging Library function identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default Aurora Imaging Library function identifier. Use this value when **MfuncErrorReport**() is called from within a callee function.	c		e	f	g	a		i		j	k	l	m	n	b
Function identifier	Specifies the identifier of a user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b

ErrorCode

Specifies a numeric code to the user-defined Aurora Imaging Library function's group of error messages. To specify the error code, combine the value in the table below with an offset. For example, M_FUNC_ERROR + 65.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For assigning a numeric code																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that no error message will be returned. Any pending internal error generated by an Aurora Imaging Library function call inside the user-defined Aurora Imaging Library function will be reset.	c		e	f	g	a		i		j	k	l	m	n	b
M_FUNC_ERROR +	Specifies a numeric code to a group of error messages. By specifying this value, you ensure that the error messages of the user-defined Aurora Imaging Library function do not conflict with Aurora Imaging Library specific errors.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_FUNC_ERROR.

You can add the following value to the above-mentioned value to specify the offset.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the offset																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
0 <= *Value* <= 99	Specifies the value of the offset.	c		e	f	g	a		i		j	k	l	m	n	b

ErrorMessage

Specifies the text of your error message to report.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the text of the error message																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*ErrorMessage*") [1]	Specifies the text of your error message; this is a null-terminated string. The error message, including the terminating null character ("\0"), must not be longer than **M_ERROR_MESSAGE_SIZE** (320) characters.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ErrorSubMessage1

Specifies the text of your first error sub-message to report.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro

Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the text of the first error sub-message																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you do not want to use this error sub-message.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*ErrorMessage*") [1]	Specifies the text of your error message; this is a null-terminated string. The error message, including the terminating null character ("\0"), must not be longer than **M_ERROR_MESSAGE_SIZE** (320) characters.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ErrorSubMessage2

Specifies the text of your second error sub-message to report.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the text of the second error sub-message																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you do not want to use this error sub-message.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*ErrorMessage*") [1]	Specifies the text of your second error sub-message. For details on creating appropriate error sub-messages, see the description of the ErrorSubMessage1 parameter.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ErrorSubMessage3

Specifies the text of your third error sub-message to report.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the text of the third error sub-message																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you do not want to use this error sub-message.	c		e	f	g	a		i		j	k	l	m	n	b

AIL_TEXT("*ErrorMessage*") [1]	Specifies the text of your third error sub-message. For details on creating appropriate error sub-messages, see the description of the ErrorSubMessage1 parameter.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Return value

The returned value is **M_NULL** if an error occurred during the error log operation; otherwise, not **M_NULL**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncFree

Synopsis

Free an Aurora Imaging Library function context.

Syntax

```
void MfuncFree(
    AIL_ID ContextFuncId //in
)
```

Description

This function frees the function context allocated for a user-defined Aurora Imaging Library function. **MfuncFree**() must be the last Aurora Imaging Library function called in the caller function. Note that you must call **MfuncFree**() from the same thread as MfuncAlloc() and MfuncCall().

All Aurora Imaging Library function contexts allocated on a particular system must be freed before the system can be freed.

Parameter

ContextFuncId

Specifies the identifier of the user-defined Aurora Imaging Library function whose function context you want to free.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncFreeId

Synopsis

Free the Aurora Imaging Library identifier associated with a user-defined Aurora Imaging Library object.

Syntax

```
void MfuncFreeId(
    AIL_ID ContextFuncId,   //in
    AIL_ID UserObjectFuncId //in
)
```

Description

This function frees an Aurora Imaging Library identifier associated with a user-defined object using the MfuncAllocId() function.

Parameters

ContextFuncId

Specifies an Aurora Imaging Library function identifier. This parameter can be set to the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the Aurora Imaging Library function identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default Aurora Imaging Library function identifier. Use M_DEFAULT to free user-defined Aurora Imaging Library objects created outside of any particular user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b
Function identifier	Specifies the identifier of a user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b

UserObjectFuncId

Specifies the Aurora Imaging Library identifier of the user-defined Aurora Imaging Library object to free.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncInquire

Synopsis

Retrieve information about an Aurora Imaging Library object.

Syntax

```
AIL_INT MfuncInquire(
    AIL_ID AilObjectId,    //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

Description

This function retrieves information about the specified Aurora Imaging Library object.

Parameters

AilObjectId

Specifies the identifier of the Aurora Imaging Library object. Depending on the inquire type, this object must have been allocated using a specific Aurora Imaging Library allocation function.

InquireType

Specifies the Aurora Imaging Library object feature about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- a pointer
- AIL_BUFFER_INFO
- AIL_INT

Specifies the address in which to write the requested information. Since the **MfuncInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For different Aurora Imaging Library object types
- For user-defined Aurora Imaging Library functions
- For the script-based user-defined Aurora Imaging Library object feature

The following InquireType parameter settings can be specified for different Aurora Imaging Library object types.

Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For different Aurora Imaging Library object types			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType		Description															
	UserVarPtr - Possible values returned																
M_BUFFER_INFO		Inquires the handle of the Aurora Imaging Library buffer. This buffer must have been allocated using one of the MbufAlloc...() functions.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_BUFFER_INFO*	c		e	f	g	a		i		j	k	l	m	n	b
M_OBJECT_PTR		Inquires the address of the user-defined Aurora Imaging Library object allocated using MfuncAllocId().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of a pointer*	c		e	f	g	a		i		j	k	l	m	n	b

The following InquireType parameter settings can be specified when the object passed to AilObjectId is a user-defined Aurora Imaging Library function context allocated using MfuncAlloc() or MfuncAllocScript().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro
Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For user-defined Aurora Imaging Library functions			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType		Description															
	UserVarPtr - Possible values returned																
M_CALLER_BITNESS		Inquires the bitness of the operating system on which the function context was created.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	32	Specifies a 32-bit platform.	c		e	f	g	a		i		j	k	l	m	n	b
	64	Specifies a 64-bit platform.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_NUMBER		Inquires the number of parameters associated with the specified user-defined Aurora Imaging Library function context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_PARAM_SIZE +	Inquires the size, in bytes, of the specified parameter. For an array parameter, the size is: *Number of items in the array* X *Size of the item*. You must specify a combination value from the following table: • To determine the number of the parameter for which to receive the information	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_INFO +	Inquires the data type of the specified parameter. You must specify a combination value from the following table: • To determine the number of the parameter for which to receive the information	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_AIL_DOUBLE	Specifies that the parameter is of type *AIL_DOUBLE*.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_AIL_ID	Specifies that the parameter is of type *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_AIL_INT	Specifies that the parameter is of type *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_AIL_INT32	Specifies that the parameter is of type *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_AIL_INT64	Specifies that the parameter is of type *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PARAM_TYPE_AIL_TEXT	Specifies that the parameter is of type *AIL_TEXT*. This implies that the character encoding scheme of the *AIL_TEXT* parameter is the scheme you are currently using.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_AIL_TEXT_ASCII	Specifies that the parameter is of type *AIL_TEXT_ASCII*. This only appears when you are currently using a character encoding scheme other than ASCII, and the parameter you are inquiring is specifically in ASCII.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_AIL_TEXT_UNICODE	Specifies that the parameter is of type *AIL_TEXT_UNICODE*. This only appears when you are currently using a character encoding scheme other than unicode, and the parameter you are inquiring is specifically in unicode.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_AIL_UINT	Specifies that the parameter is of type *AIL_UINT*.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_AIL_UINT32	Specifies that the parameter is of type *AIL_UINT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_AIL_UINT64	Specifies that the parameter is of type *AIL_UINT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_ARRAY_AIL_DOUBLE	Specifies that the parameter is an array of type *AIL_DOUBLE*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PARAM_TYPE_ARRAY_AIL_ID	Specifies that the parameter is an array of type *AIL_ID*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_ARRAY_AIL_INT	Specifies that the parameter is an array of type *AIL_INT*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_ARRAY_AIL_INT32	Specifies that the parameter is an array of type *AIL_INT32*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_ARRAY_AIL_INT64	Specifies that the parameter is an array of type *AIL_INT64*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_ARRAY_AIL_UINT	Specifies that the parameter is an array of type *AIL_UINT*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PARAM_TYPE_ARRAY_AIL_UINT32	Specifies that the parameter is an array of type *AIL_UINT32*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_ARRAY_AIL_UINT64	Specifies that the parameter is an array of type *AIL_UINT64*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_AIL_TEXT	Specifies that the parameter is a constant of type *AIL_TEXT*. This implies that the character encoding scheme of the *AIL_TEXT* parameter is the scheme you are currently using.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_AIL_TEXT_ASCII	Specifies that the parameter is a constant of type *AIL_TEXT_ASCII*. This only appears when you are currently using a character encoding scheme other than ASCII, and the parameter you are inquiring is specifically in ASCII.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_AIL_TEXT_UNICODE	Specifies that the parameter is a constant of type *AIL_TEXT_UNICODE*. This only appears when you are currently using a character encoding scheme other than unicode, and the parameter you are inquiring is specifically in unicode.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PARAM_TYPE_CONST_ARRAY_AIL_DOUBLE	Specifies that the parameter is a constant array of type *AIL_DOUBLE*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_ARRAY_AIL_ID	Specifies that the parameter is a constant array of type *AIL_ID*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_ARRAY_AIL_INT	Specifies that the parameter is a constant array of type *AIL_INT*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_ARRAY_AIL_INT32	Specifies that the parameter is a constant array of type *AIL_INT32*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_ARRAY_AIL_INT64	Specifies that the parameter is a constant array of type *AIL_INT64*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PARAM_TYPE_CONST_ARRAY_AIL_UINT	Specifies that the parameter is a constant array of type *AIL_UINT*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_ARRAY_AIL_UINT32	Specifies that the parameter is a constant array of type *AIL_UINT32*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_ARRAY_AIL_UINT64	Specifies that the parameter is a constant array of type *AIL_UINT64*. Note that the pointer to the data stored in the array is unavailable after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_CONST_DATA_PTR	Specifies that the parameter is a constant data pointer. Note that the pointer to the data is invalid after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_DATA_PTR	Specifies that the parameter is a data pointer. Note that the pointer to the data is invalid after exiting the user-defined function. For future use, copy and save the data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_FILENAME	Specifies that the parameter is of type *FILENAME*. This implies that the character encoding scheme of the *FILENAME* parameter is the scheme you are currently using.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PARAM_TYPE_FILENAME_ASCII	Specifies that the parameter is of type *FILENAME_ASCII*. This only appears when you are currently using a character encoding scheme other than ASCII, and the parameter you are inquiring is specifically in ASCII.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAM_TYPE_FILENAME_UNICODE	Specifies that the parameter is of type *FILENAME_UNICODE*. This only appears when you are currently using a character encoding scheme other than unicode, and the parameter you are inquiring is specifically in unicode.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PARAM_SIZE; M_PARAM_TYPE_INFO.

You must add the following value to the above-mentioned values to determine the number of the parameter for which to receive the information.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of the parameter for which to inquire the information																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value >= 1	Specifies the number of the parameter for which to inquire the information. This value must be a positive integer, and should not exceed the number of parameters associated with the Aurora Imaging Library function context (M_PARAM_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b

The following InquireType parameter settings can be specified when the object passed to AilObjectId is a script-based user-defined Aurora Imaging Library function context allocated using MfuncAllocScript().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the script-based user-defined Aurora Imaging Library object feature																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_COMPILE	Inquires whether the script must be reloaded from disk and recompiled. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_MODIFIED	Specifies that the script is recompiled only if the file on disk has changed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ONCE	Specifies that the script is only compiled once.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEBUG_INFORMATION		Inquires whether the interpreter interface DLL generates debug information for the script. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO	Specifies that the script is compiled in release mode, using optimization flags.	c		e	f	g	a		i		j	k	l	m	n	b
	M_YES	Specifies that the script is compiled in debug mode, and that the necessary debug information is generated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DEBUG_INFORMATION_PATH +		Inquires the path of the directory in which the compiled version of the script and other files are created, if they are relevant for the scripting language. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_SCRIPT_REFERENCES		Inquires the number of DLLs that are included in the list of DLLs referred to during runtime.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of DLLs in the list.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCRIPT_REFERENCE +		Inquires the path of the specified DLL in the list of DLLs referred to during runtime. SET You must specify a combination value from the following table: • To specify the index number of the script reference to be inquired	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_SCRIPT_REFERENCE.

You must add the following value to the above-mentioned value to specify the index number of the script reference to be inquired.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index number of the script reference to be inquired.																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value >= 1	Specifies the index of the script reference about which to inquire. This value must be a positive integer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_DEBUG_INFORMATION_PATH; M_SCRIPT_REFERENCE.

You can add the following value to the above-mentioned values to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the size of a string																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_STRING_SIZE	Inquires the number of characters in the string. This number accounts for every character, including the terminating null character.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncParam

Synopsis

Register a parameter.

Syntax

```
void MfuncParam(
    AIL_ID ContextFuncId,      //in
    AIL_INT ParamIndex,        //in
    const void *ParamValuePtr, //in
    AIL_UINT ParamType,        //in
    AIL_INT NumOfItems,        //in
    AIL_INT64 AilObjectType,   //in
    AIL_INT Attribute          //in
)
```

Description

This function allows you to register a parameter of any data type to the current user-defined Aurora Imaging Library function. The **MfuncParam**() function should be called in the caller function after a call to MfuncAlloc() and before a call to MfuncCall().

MfuncParam() calls an explicitly named type-specific variation of this function. For example, to register an *AIL_INT*, it calls MfuncParamAilInt(). Note that, for execution efficiency, you can call the appropriate, explicitly named, type-specific variation of **MfuncParam**() directly. When registering parameters using the explicitly named type-specific variation of **MfuncParam**(), you must use that function's specific signature, found below.

Parameters

ContextFuncId

Specifies the identifier of the user-defined Aurora Imaging Library function.

ParamIndex

Specifies the index of the parameter within the user-defined Aurora Imaging Library function's parameter list. The index of the first parameter is considered to be one.

ParamValuePtr

Specifies the address of the array or variable to register.

> Note that for non-arrays, you must pass a reference to the variable. For arrays, you just pass the array name.

ParamType

Specifies the parameter type of the parameter in the user-defined Aurora Imaging Library function. Parameter types include data types, arrays of data types, void pointers, strings, and file names.

See the Parameter associations section for possible values that can be specified.

NumOfItems

Specifies the size of the data pointed to by the ParamValuePtr parameter.

When the data is an array, the size is the number of elements. When the data is a string, the size is the number of characters in the string. When the data is a pointer, the size is in bytes. For all other types of data, set this parameter to **M_DEFAULT**.

This value must be greater than 0 in all cases other than **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

AilObjectType

Specifies the type of the Aurora Imaging Library object when the data type of the parameter of the user-defined function is *AIL_ID*. The available object types are divided into two groups. When the specified object is of a type listed in the first group, you can specify a single or multiple object type(s) from this group. The object types of the second group cannot be used in combination with each other.

When the data type of the parameter of the user-defined function is not *AIL_ID* or an array of *AIL_ID* s, set AilObjectType to **M_NULL**.

The following value specifies the type of Aurora Imaging Library objects that the Aurora Imaging Library identifier type parameter will accept. Any value in this table can be combined with one or more values in this table. For example, if the Aurora Imaging Library object is expected to be of type M_ARRAY or M_COUNT_LIST, set the AilObjectType parameter to M_ARRAY + M_COUNT_LIST.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of Aurora Imaging Library object accepted by the Aurora Imaging Library identifier type parameter (one or multiple)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_APPLICATION	Specifies an Aurora Imaging Library application context allocated using MappAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_ARRAY	Specifies an Aurora Imaging Library array buffer allocated using MbufAlloc2d() with M_ARRAY.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUGMENTATION_RESULT	Specifies an Aurora Imaging Library augmentation result buffer allocated using MimAllocResult() with M_AUGMENTATION_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTAINER	Specifies an Aurora Imaging Library container buffer allocated using MbufAllocContainer().	c		e	f	g	a		i		j	k	l	m	n	b
M_COUNT_LIST	Specifies an Aurora Imaging Library count list result buffer allocated using MimAllocResult() with M_COUNT_LIST.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIGITIZER	Specifies an Aurora Imaging Library digitizer allocated using MdigAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

M_DISPLAY	Specifies an Aurora Imaging Library display allocated using MdispAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_EVENT	Specifies an Aurora Imaging Library event allocated using MthrAlloc() with M_EVENT.	c		e	f	g	a		i		j	k	l	m	n	b
M_EVENT_LIST	Specifies an Aurora Imaging Library event list result buffer allocated using MimAllocResult() with M_EVENT_LIST.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTREME_LIST	Specifies an Aurora Imaging Library extreme list result buffer allocated using MimAllocResult() with M_EXTREME_LIST.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIND_ORIENTATION_LIST	Specifies an Aurora Imaging Library find orientation list result buffer allocated using MimAllocResult() with M_FIND_ORIENTATION_LIST.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_CONTEXT	Specifies an Aurora Imaging Library 2D graphics context allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST	Specifies an Aurora Imaging Library 2D graphics list allocated using MgraAllocList().	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_LIST	Specifies an Aurora Imaging Library histogram list result buffer allocated using MimAllocResult() with M_HIST_LIST.	c		e	f	g	a		i		j	k	l	m	n	b
M_HTTP_SERVER	Specifies an HTTP server allocated using MobjAlloc() with M_HTTP_SERVER.	c		e	f	g	a		i		j	k	l	m	n	b
M_IM_CONTEXT	Specifies an Aurora Imaging Library image processing context allocated using MimAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE	Specifies an Aurora Imaging Library image buffer allocated using MbufAllocColor() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_KERNEL	Specifies an Aurora Imaging Library kernel buffer allocated using MbufAlloc2d() with M_KERNEL.	c		e	f	g	a		i		j	k	l	m	n	b

M_LOCATE_DIFFERENCE_RESULT	Specifies an Aurora Imaging Library locate difference result buffer allocated using MimAllocResult() with M_LOCATE_DIFFERENCE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCATE_PEAK_1D_RESULT	Specifies an Aurora Imaging Library locate 1D peak result buffer allocated using MimAllocResult() with M_LOCATE_PEAK_1D_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT	Specifies an Aurora Imaging Library LUT buffer allocated using MbufAlloc2d() with M_LUT.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESSAGE_MAILBOX	Specifies an Aurora Imaging Library message mailbox allocated using MobjAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_PROJ_LIST	Specifies an Aurora Imaging Library project list result buffer allocated using MimAllocResult() with M_PROJ_LIST.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEQUENCE_CONTEXT	Specifies an Aurora Imaging Library sequence context allocated using MseqAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_STATISTICS_RESULT	Specifies an Aurora Imaging Library MimStatCalculate() result buffer allocated using MimAllocResult() with M_STATISTICS_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRUCT_ELEMENT	Specifies an Aurora Imaging Library structuring element buffer allocated using MbufAlloc2d() with M_STRUCT_ELEMENT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SYS_IO_CONTEXT	Specifies an Aurora Imaging Library system I/O context allocated using MsysIoAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_SYSTEM	Specifies an Aurora Imaging Library system context allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_TRANSFORM_RESULT	Specifies an Aurora Imaging Library wavelet transform result buffer allocated using MimAllocResult() with M_WAVELET_TRANSFORM_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

The following value specifies the type of Aurora Imaging Library objects that the Aurora Imaging Library

identifier type parameter will accept. No value in this table can be combined with any other value in this table, though a value in this table can be combined with one or more values from the above table.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of Aurora Imaging Library object accepted by the Aurora Imaging Library identifier type parameter (only one)																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_3D_DISPLAY	Specifies an Aurora Imaging Library 3D display allocated using M3ddispAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISPLAY_PICKING_AREA_CONTEXT	Specifies an Aurora Imaging Library picking area context allocated using M3ddispAlloc() with M_PICKING_AREA_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISPLAY_PICKING_AREA_RESULT	Specifies an Aurora Imaging Library picking area result buffer allocated using M3ddispAllocResult() with M_PICKING_AREA_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISPLAY_PICKING_CONTEXT	Specifies an Aurora Imaging Library picking context allocated using M3ddispAlloc() with M_PICKING_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISPLAY_PICKING_RESULT	Specifies an Aurora Imaging Library picking result buffer allocated using M3ddispAllocResult() with M_PICKING_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_GRAPHIC_LIST	Specifies an Aurora Imaging Library 3D graphics list allocated using M3dgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_3DBLOB_CALCULATE_CONTEXT	Specifies an Aurora Imaging Library 3D blob context that can be used with M3dblobCalculate(), allocated using M3dblobAlloc() with M_CALCULATE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DBLOB_DRAW_3D_CONTEXT	Specifies an Aurora Imaging Library 3D blob context that can be used with M3dblobDraw3d(), allocated using M3dblobAlloc() with M_DRAW_3D_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

M_3DBLOB_SEGMENTATION_CONTEXT	Specifies an Aurora Imaging Library 3D blob context that can be used with M3dblobSegment(), allocated using M3dblobAlloc() with M_SEGMENTATION_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DBLOB_SEGMENTATION_RESULT	Specifies an Aurora Imaging Library 3D blob result buffer for segmentation results, allocated using M3dblobAllocResult() with M_SEGMENTATION_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DGEO_GEOMETRY	Specifies an Aurora Imaging Library 3D geometry object allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DGEO_TRANSFORMATION_MATRIX	Specifies an Aurora Imaging Library 3D transformation matrix object allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_CALCULATE_MAP_SIZE_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimCalculateMapSize(), allocated using M3dimAlloc() with M_CALCULATE_MAP_SIZE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_FILL_GAPS_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimFillGaps(), allocated using M3dimAlloc() with M_FILL_GAPS_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_FILTER_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimFilter(), allocated using M3dimAlloc() with M_FILTER_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_FIND_TRANSFORMATION_RESULT	Specifies an Aurora Imaging Library 3D image processing result buffer for find transformation results, allocated using M3dimAllocResult() with M_FIND_TRANSFORMATION_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_LATTICE_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimLattice(), allocated using M3dimAlloc() with M_LATTICE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

M_3DIM_LATTICE_RESULT	Specifies an Aurora Imaging Library 3D image processing result buffer for lattice results, allocated using M3dimAllocResult() with M_LATTICE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_MESH_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimMesh(), allocated using M3dimAlloc() with M_MESH_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_NORMALS_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimNormals(), allocated using M3dimAlloc() with M_NORMALS_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_OUTLIERS_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimOutliers(), allocated using M3dimAlloc() with M_OUTLIERS_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_PROFILE_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimProfileEx(), allocated using M3dimAlloc() with M_PROFILE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_PROFILE_RESULT	Specifies an Aurora Imaging Library 3D image processing result buffer for profile results, allocated using M3dimAllocResult() with M_PROFILE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_PROJECT_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimProjectEx(), allocated using M3dimAlloc() with M_PROJECT_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_REMAP_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimRemapDepthMap(), allocated using M3dimAlloc() with M_REMAP_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_STATISTICS_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimStat(), allocated using M3dimAlloc() with M_STATISTICS_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

M_3DIM_STATISTICS_RESULT	Specifies an Aurora Imaging Library 3D image processing result buffer used to store M3dimStat() results, and is allocated using M3dimAllocResult() with M_STATISTICS_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_STITCH_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimMerge(), allocated using M3dimAlloc() with M_STITCH_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_SUBSAMPLE_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimSample(), allocated using M3dimAlloc() with M_SUBSAMPLE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DIM_SURFACE_SAMPLE_CONTEXT	Specifies an Aurora Imaging Library 3D image processing context that can be used with M3dimSample(), allocated using M3dimAlloc() with M_SURFACE_SAMPLE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMAP_ALIGN_CONTEXT	Specifies an Aurora Imaging Library 3D align context allocated using M3dmapAlloc() with M_ALIGN_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMAP_ALIGN_RESULT	Specifies an Aurora Imaging Library 3D align result buffer allocated using M3dmapAllocResult() with M_ALIGN_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMAP_DEPTH_CORRECTED_DATA	Specifies an Aurora Imaging Library 3D reconstruction result buffer allocated using M3dmapAllocResult() with M_DEPTH_CORRECTED_DATA.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMAP_DRAW_3D_CONTEXT	Specifies an Aurora Imaging Library 3D draw context allocated using M3dmapAlloc() with M_DRAW_3D_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMAP_LASER_CALIBRATION_DATA	Specifies an Aurora Imaging Library 3D reconstruction result buffer allocated using M3dmapAllocResult() with M_LASER_CALIBRATION_DATA.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMAP_LASER_CONTEXT	Specifies an Aurora Imaging Library 3D reconstruction context allocated using M3dmapAlloc() with M_LASER.	c		e	f	g	a		i		j	k	l	m	n	b

M_3DMAP_POINT_CLOUD_RESULT	Specifies an Aurora Imaging Library 3D reconstruction result buffer allocated using M3dmapAllocResult() with M_POINT_CLOUD_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMEAS_DRAW_3D_PATH_CONTEXT	Specifies an Aurora Imaging Library draw path 3D measurement context allocated using M3dmeasAlloc() with M_DRAW_3D_PATH_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMEAS_DRAW_3D_PROFILE_CONTEXT	Specifies an Aurora Imaging Library draw profile 3D measurement context allocated using M3dmeasAlloc() with M_DRAW_3D_PROFILE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMEAS_DRAW_3D_TEMPLATE_CONTEXT	Specifies an Aurora Imaging Library draw template 3D measurement context allocated using M3dmeasAlloc() with M_DRAW_3D_TEMPLATE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMEAS_FIND_MARKER_PATH_CONTEXT	Specifies an Aurora Imaging Library path 3D measurement context allocated using M3dmeasAlloc() with M_FIND_MARKER_PATH_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMEAS_FIND_MARKER_PATH_RESULT	Specifies an Aurora Imaging Library path 3D measurement result allocated using M3dmeasAllocResult() with M_FIND_MARKER_PATH_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMEAS_FIND_MARKER_PROFILE_CONTEXT	Specifies an Aurora Imaging Library profile 3D measurement context allocated using M3dmeasAlloc() with M_FIND_MARKER_PROFILE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMEAS_FIND_MARKER_PROFILE_RESULT	Specifies an Aurora Imaging Library profile 3D measurement result allocated using M3dmeasAllocResult() with M_FIND_MARKER_PROFILE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMEAS_FIND_MARKER_TEMPLATE_CONTEXT	Specifies an Aurora Imaging Library template 3D measurement context allocated using M3dmeasAlloc() with M_FIND_MARKER_TEMPLATE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMEAS_FIND_MARKER_TEMPLATE_RESULT	Specifies an Aurora Imaging Library template 3D measurement result allocated using M3dmeasAllocResult() with M_FIND_MARKER_TEMPLATE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

M_3DMEAS_FIT_CONTEXT	Specifies an Aurora Imaging Library fit 3D measurement context allocated using M3dmeasAlloc() with M_FIT_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMET_CALCULATE_RESULT	Specifies an Aurora Imaging Library calculate 3D metrology result buffer allocated using M3dmetAllocResult() with M_CALCULATE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMET_DISTANCE_CONTEXT	Specifies an Aurora Imaging Library distance 3D metrology context allocated using M3dmetAlloc() with M_DISTANCE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMET_DRAW_3D_CONTEXT	Specifies an Aurora Imaging Library draw 3D metrology context allocated using M3dmetAlloc() with M_DRAW_3D_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMET_FIT_CONTEXT	Specifies an Aurora Imaging Library fit 3D metrology context allocated using M3dmetAlloc() with M_FIT_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMET_FIT_RESULT	Specifies an Aurora Imaging Library fit 3D metrology result buffer allocated using M3dmetAllocResult() with M_FIT_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMET_STATISTICS_CONTEXT	Specifies an Aurora Imaging Library statistics 3D metrology context allocated using M3dmetAlloc() with M_STATISTICS_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMET_STATISTICS_RESULT	Specifies an Aurora Imaging Library statistics 3D metrology result buffer allocated using M3dmetAllocResult() with M_STATISTICS_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMET_VOLUME_CONTEXT	Specifies an Aurora Imaging Library calculate volume 3D metrology context allocated using M3dmetAlloc() with M_VOLUME_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_DRAW_3D_CONTEXT	Specifies an Aurora Imaging Library draw 3D model finder context that can be allocated using M3dmodAlloc() with M_DRAW_3D_GEOMETRIC_CONTEXT (to draw occurrences of geometric models).	c		e	f	g	a		i		j	k	l	m	n	b

M_3DMOD_DRAW_3D_SURFACE_CONTEXT	Specifies an Aurora Imaging Library draw 3D model finder context that can be allocated using M3dmodAlloc() with M_DRAW_3D_SURFACE_CONTEXT (to draw occurrences of surface models).	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_BOX_CONTEXT	Specifies an Aurora Imaging Library box 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_BOX_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_BOX_RESULT	Specifies an Aurora Imaging Library box 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_BOX_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_CYLINDER_CONTEXT	Specifies an Aurora Imaging Library cylinder 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_CYLINDER_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_CYLINDER_RESULT	Specifies an Aurora Imaging Library cylinder 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_CYLINDER_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_PLANAR_SURFACE_CONTEXT	Specifies an Aurora Imaging Library planar surface 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_PLANAR_SURFACE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_PLANAR_SURFACE_RESULT	Specifies an Aurora Imaging Library planar surface 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_PLANAR_SURFACE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_RECTANGULAR_PLANE_CONTEXT	Specifies an Aurora Imaging Library rectangular plane 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_RECTANGULAR_PLANE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_RECTANGULAR_PLANE_RESULT	Specifies an Aurora Imaging Library rectangular plane 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_RECTANGULAR_PLANE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

M_3DMOD_FIND_SPHERE_CONTEXT	Specifies an Aurora Imaging Library sphere 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_SPHERE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_SPHERE_RESULT	Specifies an Aurora Imaging Library sphere 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_SPHERE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_SURFACE_CONTEXT	Specifies an Aurora Imaging Library surface 3D model finder context that can be allocated using M3dmodAlloc() with M_FIND_SURFACE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DMOD_FIND_SURFACE_RESULT	Specifies an Aurora Imaging Library surface 3D model finder result buffer allocated using M3dmodAllocResult() with M_FIND_SURFACE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DREG_DRAW_3D_CONTEXT	Specifies an Aurora Imaging Library draw 3D registration context that can be used with M3dregDraw3d(), allocated using M3dregAlloc() with M_DRAW_3D_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DREG_PAIRWISE_REGISTRATION_CONTEXT	Specifies an Aurora Imaging Library pairwise 3D registration context allocated using M3dregAlloc() with M_PAIRWISE_REGISTRATION_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_3DREG_PAIRWISE_REGISTRATION_RESULT	Specifies an Aurora Imaging Library pairwise 3D registration result buffer allocated using M3dregAllocResult() with M_PAIRWISE_REGISTRATION_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_AGM_EDGE_BASED_FIND_CONTEXT	Specifies an Aurora Imaging Library Advanced Geometric Matching context allocated using MagmAlloc() with M_GLOBAL_EDGE_BASED_FIND.	c		e	f	g	a		i		j	k	l	m	n	b
M_AGM_EDGE_BASED_FIND_RESULT	Specifies an Aurora Imaging Library Advanced Geometric Matching result buffer allocated using MagmAllocResult() with M_GLOBAL_EDGE_BASED_FIND_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_AGM_EDGE_BASED_TRAIN_CONTEXT	Specifies an Aurora Imaging Library Advanced Geometric Matching context allocated using MagmAlloc() with M_GLOBAL_EDGE_BASED_TRAIN.	c		e	f	g	a		i		j	k	l	m	n	b

M_AGM_EDGE_BASED_TRAIN_RESULT	Specifies an Aurora Imaging Library Advanced Geometric Matching result buffer allocated using MagmAllocResult() with M_GLOBAL_EDGE_BASED_TRAIN_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_BEAD_CONTEXT	Specifies an Aurora Imaging Library bead context allocated using MbeadAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_BEAD_RESULT	Specifies an Aurora Imaging Library bead result buffer allocated using MbeadAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_BLOB_CONTEXT	Specifies an Aurora Imaging Library blob analysis context allocated using MblobAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_BLOB_RESULT	Specifies an Aurora Imaging Library blob analysis result buffer allocated using MblobAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_CAL_CALCULATE_HAND_EYE_CONTEXT	Specifies an Aurora Imaging Library calculate hand-eye context (hand-eye calibration) allocated using McalAlloc() with M_CALCULATE_HAND_EYE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CAL_CALCULATE_HAND_EYE_RESULT	Specifies an Aurora Imaging Library calculate hand-eye result buffer allocated using McalAllocResult() with M_CALCULATE_HAND_EYE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CAL_CONTEXT	Specifies an Aurora Imaging Library camera calibration context allocated using McalAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_CAL_DRAW_3D_CONTEXT	Specifies an Aurora Imaging Library 3D draw calibration context allocated using McalAlloc() with M_DRAW_3D_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CAL_FIXTURING_OFFSET	Specifies an Aurora Imaging Library fixturing offset object allocated using McalAlloc() with M_FIXTURING_OFFSET.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_CLASSIFIER_ANO_PREDEFINED_CONTEXT	Specifies an Aurora Imaging Library predefined anomaly detection classifier context, allocated using MclassAlloc() with M_CLASSIFIER_ANO_PREDEFINED.	c		e	f	g	a		i		j	k	l	m	n	b

M_CLASS_CLASSIFIER_CNN_PREDEFINED_CONTEXT	Specifies an Aurora Imaging Library predefined CNN classifier context, allocated using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_CLASSIFIER_DET_PREDEFINED_CONTEXT	Specifies an Aurora Imaging Library predefined object detection classifier context, allocated using MclassAlloc() with M_CLASSIFIER_DET_PREDEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_CLASSIFIER_ONNX_CONTEXT	Specifies an Aurora Imaging Library ONNX classifier context, allocated using MclassAlloc() with M_CLASSIFIER_ONNX.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_CLASSIFIER_SEG_PREDEFINED_CONTEXT	Specifies an Aurora Imaging Library predefined segmentation classifier context, allocated using MclassAlloc() with M_CLASSIFIER_SEG_PREDEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_CLASSIFIER_TREE_ENSEMBLE_CONTEXT	Specifies an Aurora Imaging Library tree ensemble classifier context, allocated using MclassAlloc() with M_CLASSIFIER_TREE_ENSEMBLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_DATASET_FEATURES	Specifies an Aurora Imaging Library features dataset context, allocated using MclassAlloc() with M_DATASET_FEATURES.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_DATASET_IMAGES	Specifies an Aurora Imaging Library images dataset context, allocated using MclassAlloc() with M_DATASET_IMAGES.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_PREDICT_ANO_RESULT	Specifies an Aurora Imaging Library anomaly detection prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_ANO_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_PREDICT_CNN_RESULT	Specifies an Aurora Imaging Library CNN prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_CNN_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_PREDICT_DET_RESULT	Specifies an Aurora Imaging Library object detection prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_DET_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_PREDICT_ONNX_RESULT	Specifies an ONNX prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_ONNX_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

M_CLASS_PREDICT_SEG_RESULT	Specifies an Aurora Imaging Library segmentation prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_SEG_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_PREDICT_TREE_ENSEMBLE_RESULT	Specifies an Aurora Imaging Library tree ensemble prediction result buffer, allocated using MclassAllocResult() with M_PREDICT_TREE_ENSEMBLE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_PREPARE_IMAGES_CNN	Specifies an Aurora Imaging Library CNN data preparation context, allocated using MclassAlloc() with M_PREPARE_IMAGES_CNN.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_PREPARE_IMAGES_DET	Specifies an Aurora Imaging Library object detection data preparation context, allocated using MclassAlloc() with M_PREPARE_IMAGES_DET.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_PREPARE_IMAGES_SEG	Specifies an Aurora Imaging Library segmentation data preparation context, allocated using MclassAlloc() with M_PREPARE_IMAGES_SEG.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_STAT_ANO_CONTEXT	Specifies an Aurora Imaging Library statistics classification context for anomaly detection, allocated using MclassAlloc() with M_STAT_ANO.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_STAT_ANO_RESULT	Specifies an Aurora Imaging Library statistics classification result buffer for anomaly detection, allocated using MclassAllocResult() with M_STAT_ANO_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_STAT_CNN_CONTEXT	Specifies an Aurora Imaging Library statistics classification context for image classification, allocated using MclassAlloc() with M_STAT_CNN.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_STAT_CNN_RESULT	Specifies an Aurora Imaging Library statistics classification result buffer for image classification, allocated using MclassAllocResult() with M_STAT_CNN_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_STAT_DET_CONTEXT	Specifies an Aurora Imaging Library statistics classification context for object detection, allocated using MclassAlloc() with M_STAT_DET.	c		e	f	g	a		i		j	k	l	m	n	b

M_CLASS_STAT_DET_RESULT	Specifies an Aurora Imaging Library statistics classification result buffer for object detection, allocated using MclassAllocResult() with M_STAT_DET_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_STAT_SEG_CONTEXT	Specifies an Aurora Imaging Library statistics classification context for segmentation, allocated using MclassAlloc() with M_STAT_SEG.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_STAT_SEG_RESULT	Specifies an Aurora Imaging Library statistics classification result buffer for segmentation, allocated using MclassAllocResult() with M_STAT_SEG_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_STAT_TREE_ENSEMBLE_CONTEXT	Specifies an Aurora Imaging Library statistics classification context for feature classification, allocated using MclassAlloc() with M_STAT_TREE_ENSEMBLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_STAT_TREE_ENSEMBLE_RESULT	Specifies an Aurora Imaging Library statistics classification result buffer for feature classification, allocated using MclassAllocResult() with M_STAT_TREE_ENSEMBLE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_TRAIN_ANO_CONTEXT	Specifies an Aurora Imaging Library anomaly detection training context, allocated using MclassAlloc() with M_TRAIN_ANO.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_TRAIN_ANO_RESULT	Specifies an Aurora Imaging Library anomaly detection training result buffer, allocated using MclassAllocResult() with M_TRAIN_ANO_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_TRAIN_CNN_CONTEXT	Specifies an Aurora Imaging Library CNN training context, allocated using MclassAlloc() with M_TRAIN_CNN.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_TRAIN_CNN_RESULT	Specifies an Aurora Imaging Library CNN training result buffer, allocated using MclassAllocResult() with M_TRAIN_CNN_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_TRAIN_DET_CONTEXT	Specifies an Aurora Imaging Library object detection training context, allocated using MclassAlloc() with M_TRAIN_DET.	c		e	f	g	a		i		j	k	l	m	n	b

M_CLASS_TRAIN_DET_RESULT	Specifies an Aurora Imaging Library object detection training result buffer, allocated using MclassAllocResult() with M_TRAIN_DET_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_TRAIN_SEG_CONTEXT	Specifies an Aurora Imaging Library segmentation training context, allocated using MclassAlloc() with M_TRAIN_SEG.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_TRAIN_SEG_RESULT	Specifies an Aurora Imaging Library segmentation training result buffer, allocated using MclassAllocResult() with M_TRAIN_CNN_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_TRAIN_TREE_ENSEMBLE_CONTEXT	Specifies an Aurora Imaging Library tree ensemble training context, allocated using MclassAlloc() with M_TRAIN_TREE_ENSEMBLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_TRAIN_TREE_ENSEMBLE_RESULT	Specifies an Aurora Imaging Library tree ensemble training result buffer, allocated using MclassAllocResult() with M_TRAIN_TREE_ENSEMBLE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_CONTEXT	Specifies an Aurora Imaging Library code context allocated using McolAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_DETECT_RESULT	Specifies an Aurora Imaging Library code detect result buffer allocated using McodeAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_GRADE_RESULT	Specifies an Aurora Imaging Library code grade result buffer allocated using McodeAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_MODEL	Specifies an Aurora Imaging Library code model allocated using McodeModel() with M_ADD.	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_READ_RESULT	Specifies an Aurora Imaging Library code read result buffer allocated using McodeAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_TRAIN_RESULT	Specifies an Aurora Imaging Library code train result buffer allocated using McodeAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_CODE_WRITE_RESULT	Specifies an Aurora Imaging Library code write result buffer allocated using McodeAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b

M_COL_MATCH_CONTEXT	Specifies an Aurora Imaging Library color analysis context (for matching) allocated using McolAlloc() with M_COLOR_MATCHING.	c		e	f	g	a		i		j	k	l	m	n	b
M_COL_MATCH_RESULT	Specifies an Aurora Imaging Library color analysis result buffer (for matching) allocated using McolAllocResult() with M_COLOR_MATCHING_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_COL_RELATIVE_CALIBRATION_CONTEXT	Specifies an Aurora Imaging Library relative color calibration context allocated using McolAlloc() with M_COLOR_CALIBRATION_RELATIVE.	c		e	f	g	a		i		j	k	l	m	n	b
M_COM_CONTEXT	Specifies an Aurora Imaging Library industrial communication context allocated using McomAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_DLOCR_DATASET	Specifies an Aurora Imaging Library Deep Learning OCR dataset allocated using MdlocrAllocDataset().	c		e	f	g	a		i		j	k	l	m	n	b
M_DLOCR_FINETUNE_CONTEXT	Specifies an Aurora Imaging Library Deep Learning OCR fine-tuning context allocated using MdlocrAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_DLOCR_FINETUNE_RESULT	Specifies an Aurora Imaging Library Deep Learning OCR fine-tuning result buffer allocated using MdlocrAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_DLOCR_READ_CONTEXT	Specifies an Aurora Imaging Library Deep Learning OCR read context allocated using MdlocrAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_DLOCR_READ_RESULT	Specifies an Aurora Imaging Library Deep Learning OCR read result buffer allocated using MdlocrAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_DMR_CONTEXT	Specifies an Aurora Imaging Library SureDotOCR context allocated using MdmrAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_DMR_RESULT	Specifies an Aurora Imaging Library SureDotOCR result buffer allocated using MdmrAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGE_CONTOUR	Specifies an Aurora Imaging Library edge contour context allocated using MedgeAlloc() with M_CONTOUR.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_CREST	Specifies an Aurora Imaging Library edge crest context allocated using MedgeAlloc() with M_CREST.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_RESULT	Specifies an Aurora Imaging Library edge result buffer allocated using MedgeAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAS_CONTEXT	Specifies an Aurora Imaging Library measurement context allocated using MmeasAllocContext().	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAS_MARKER	Specifies an Aurora Imaging Library measurement marker allocated using MmeasAllocMarker().	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAS_RESULT	Specifies an Aurora Imaging Library measurement result buffer allocated using MmeasAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_MET_CONTEXT	Specifies an Aurora Imaging Library metrology context allocated using MmetAlloc() with M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_MET_DERIVED_GEOMETRY_REGION	Specifies an Aurora Imaging Library metrology derived geometry region object allocated using MmetAlloc() with M_DERIVED_GEOMETRY_REGION.	c		e	f	g	a		i		j	k	l	m	n	b
M_MET_RESULT	Specifies an Aurora Imaging Library metrology result buffer allocated using MmetAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_MOD_GEOMETRIC	Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_GEOMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOD_GEOMETRIC_CONTROLLED	Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_GEOMETRIC_CONTROLLED.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOD_RESULT	Specifies an Aurora Imaging Library Model Finder result buffer allocated using MmodAllocResult() with M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b

M_MOD_SHAPE_CIRCLE_CONTEXT	Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_CIRCLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOD_SHAPE_ELLIPSE_CONTEXT	Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_ELLIPSE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOD_SHAPE_RECTANGLE_CONTEXT	Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_RECTANGLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOD_SHAPE_RESULT	Specifies an Aurora Imaging Library Model Finder result buffer allocated using MmodAllocResult() with M_SHAPE_....	c		e	f	g	a		i		j	k	l	m	n	b
M_MOD_SHAPE_SEGMENT_CONTEXT	Specifies an Aurora Imaging Library Model Finder context allocated using MmodAlloc() with M_SHAPE_SEGMENT.	c		e	f	g	a		i		j	k	l	m	n	b
M_MUTEX	Specifies an Aurora Imaging Library mutex allocated using MthrAlloc() with M_MUTEX.	c		e	f	g	a		i		j	k	l	m	n	b
M_OCR_FONT	Specifies an Aurora Imaging Library character recognition font context allocated using MocrAllocFont().	c		e	f	g	a		i		j	k	l	m	n	b
M_OCR_RESULT	Specifies an Aurora Imaging Library character recognition result buffer allocated using MocrAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_PAT_CONTEXT	Specifies an Aurora Imaging Library pattern matching context allocated using MpatAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_PAT_RESULT	Specifies an Aurora Imaging Library pattern matching result buffer allocated using MpatAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b
M_REG_DFF_CONTEXT	Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_DEPTH_FROM_FOCUS.	c		e	f	g	a		i		j	k	l	m	n	b
M_REG_DFF_RESULT	Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_DEPTH_FROM_FOCUS_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

M_REG_EDOF_CONTEXT	Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_EXTENDED_DEPTH_OF_FIELD.	c		e	f	g	a		i		j	k	l	m	n	b
M_REG_EDOF_RESULT	Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_EXTENDED_DEPTH_OF_FIELD_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_REG_HDR_CONTEXT	Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_HIGH_DYNAMIC_RANGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_REG_HDR_RESULT	Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_HIGH_DYNAMIC_RANGE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_REG_PHOTOMETRIC_STEREO_CONTEXT	Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_PHOTOMETRIC_STEREO.	c		e	f	g	a		i		j	k	l	m	n	b
M_REG_PHOTOMETRIC_STEREO_RESULT	Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_PHOTOMETRIC_STEREO_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_REG_STITCHING_CONTEXT	Specifies an Aurora Imaging Library registration context allocated using MregAlloc() with M_STITCHING.	c		e	f	g	a		i		j	k	l	m	n	b
M_REG_STITCHING_RESULT	Specifies an Aurora Imaging Library registration result buffer allocated using MregAllocResult() with M_STITCHING_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SELECTABLE_THREAD	Specifies an Aurora Imaging Library selectable thread allocated using MthrAlloc() with M_SELECTABLE_THREAD.	c		e	f	g	a		i		j	k	l	m	n	b
M_STR_CONTEXT	Specifies an Aurora Imaging Library string context allocated using MstrAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_STR_RESULT	Specifies an Aurora Imaging Library string result buffer allocated using MstrAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b

M_SYSTEM_THREAD	Specifies an Aurora Imaging Library thread allocated using MthrAlloc() with M_THREAD.	c		e	f	g	a		i		j	k	l	m	n	b
M_THREAD	Specifies an Aurora Imaging Library thread context object allocated using MthrAlloc() with M_THREAD.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_OBJECT_1 +	Specifies an Aurora Imaging Library object from group one of the user-defined object types allocated using MfuncAllocId() with M_USER_OBJECT_1. You must specify a combination value from the following table: • To set the offset	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_OBJECT_2 +	Specifies an Aurora Imaging Library object from group two of the user-defined object types allocated using MfuncAllocId() with M_USER_OBJECT_2. You must specify a combination value from the following table: • To set the offset	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_USER_OBJECT_1; M_USER_OBJECT_2.

You must add the following value to the above-mentioned values to set the offset.

The offset allows you to distinguish between the different object types of the same group (for example, M_USER_OBJECT_1 + 0x0001).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For distinguishing between the different object types																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value	Specifies the offset within the selected object type group. The value must have only one of its 16 least significant bits set.	c		e	f	g	a		i		j	k	l	m	n	b

Attribute

Specifies whether the parameter will be used for data input or to store new data.

> When registering a const array type parameter, Attribute must be set to M_IN.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ParamType**, **NumOfItems**, and **Attribute** parameters.

- For registering a primitive data-type parameter in an Aurora Imaging Library user-defined function
- For registering a primitive data-type array parameter in an Aurora Imaging Library user-defined function
- For registering a const array of primitive data-types parameter in an Aurora Imaging Library user-defined function
- For registering a string or file name type parameter in an Aurora Imaging Library user-defined function
- For registering a pointer type parameter in an Aurora Imaging Library user-defined function
- For registering an Aurora Imaging Library identifier type parameter in an Aurora Imaging Library user-defined function

The following values specify to call type-specific variations of **MfuncParam**() that register parameters that handle primitive Aurora Imaging Library data types, such as, *AIL_DOUBLE*, *AIL_INT*, and *AIL_UINT64*, as well as strings, void pointers and file names.

Unless otherwise specified, when specifying values from this table, you must set the NumOfItems parameter to **M_DEFAULT**, the AilObjectType parameter to **M_NULL**, and the Attribute parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For registering a primitive data-type parameter in an Aurora Imaging Library user-defined function			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ParamType		Description															
	NumOfItems																
	Attribute																
M_PARAM_TYPE_AIL_DOUBLE		Specifies to call MfuncParamAilDouble(), which then registers a type *AIL_DOUBLE* parameter for the current user-defined Aurora Imaging Library function.															
M_PARAM_TYPE_AIL_INT		Specifies to call MfuncParamAilInt(), which then registers a type *AIL_INT* parameter for the current user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_AIL_INT32		Specifies to call MfuncParamAilInt32(), which then registers a type *AIL_INT32* parameter for the current user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_AIL_INT64		Specifies to call MfuncParamAilInt64(), which then registers a type *AIL_INT64* parameter for the current user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_AIL_UINT		Specifies to call MfuncParamAilUint(), which then registers a type *AIL_UINT* parameter for the current user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b

M_PARAM_TYPE_AIL_UINT32	Specifies to call MfuncParamAilUint32(), which then registers a type *AIL_UINT32* parameter for the current user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_AIL_UINT64	Specifies to call MfuncParamAilUint64(), which then registers a type *AIL_INT64* parameter for the current user-defined Aurora Imaging Library function.	c		e	f	g	a		i		j	k	l	m	n	b

The following values specify to call type-specific variations of **MfuncParam**() that register parameters that handle arrays of primitive Aurora Imaging Library data types, such as, *AIL_DOUBLE*, *AIL_INT*, and *AIL_UINT64*.

When specifying values from this table, you must set the NumOfItems parameter to the number of items in the array, and you must set the AilObjectType parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For registering a primitive data-type array parameter in an Aurora Imaging Library user-defined function																		
ParamType			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	NumOfItems																	
	Attribute																	
M_PARAM_TYPE_ARRAY_AIL_DOUBLE			Specifies to call MfuncParamArrayAilDouble(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles an array of type *AIL_DOUBLE*.															
	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the array passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_ARRAY_AIL_INT			Specifies to call MfuncParamArrayAilInt(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles an array of type *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b

	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the array passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_ARRAY_AIL_INT32			Specifies to call MfuncParamArrayAilInt32(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles an array of type *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the array passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_ARRAY_AIL_INT64			Specifies to call MfuncParamArrayAilInt64(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles an array of type *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the array passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b

M_PARAM_TYPE_ARRAY_AIL_UINT			Specifies to call MfuncParamArrayAilUint(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles an array of type *AIL_UINT*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the array passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_ARRAY_AIL_UINT32			Specifies to call MfuncParamArrayAilUint32(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles an array of type *AIL_UINT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the array passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_ARRAY_AIL_UINT64			Specifies to call MfuncParamArrayAilUint64(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles an array of type *AIL_UINT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b

		M_OUT	Specifies that the array passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b

The following values specify to call type-specific variations of **MfuncParam**() that register parameters that handle const arrays of primitive Aurora Imaging Library data types, such as, *AIL_DOUBLE*, *AIL_INT*, and *AIL_UINT64*, as well as const pointers and const strings.

Unless otherwise specified, when specifying values from this table, you must set the NumOfItems parameter to the number of items in the array, and you must set the AilObjectType parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For registering a const array of primitive data-types parameter in an Aurora Imaging Library user-defined function																		
ParamType			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	NumOfItems																	
	Attribute																	
M_PARAM_TYPE_CONST_ARRAY_AIL_DOUBLE			Specifies to call MfuncParamConstArrayAilDouble(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const array of type *AIL_DOUBLE*.															
	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_CONST_ARRAY_AIL_INT			Specifies to call MfuncParamConstArrayAilInt(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const array of type *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b

M_PARAM_TYPE_CONST_ARRAY_AIL_INT32			Specifies to call MfuncParamConstArrayAilInt32(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const array of type *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_CONST_ARRAY_AIL_INT64			Specifies to call MfuncParamConstArrayAilInt64(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const array of type *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_CONST_ARRAY_AIL_UINT			Specifies to call MfuncParamConstArrayAilUint(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const array of type *AIL_UINT*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_CONST_ARRAY_AIL_UINT32			Specifies to call MfuncParamConstArrayAilUint32(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const array of type *AIL_UINT32*.	c		e	f	g	a		i		j	k	l	m	n	b

	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_CONST_ARRAY_AIL_UINT64			Specifies to call MfuncParamConstArrayAilUint64(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const array of type *AIL_UINT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b

The following values specify to call type-specific variations of **MfuncParam**() that register a string or file name parameter.

Unless otherwise specified, when specifying values from this table, you must set the AilObjectType parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For registering a string or file name type parameter in an Aurora Imaging Library user-defined function																		
ParamType			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	NumOfItems																	
	Attribute																	
M_PARAM_TYPE_AIL_TEXT			Specifies to call MfuncParamAilText(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a string.															
	NumOfItems		Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies for Aurora Imaging Library to auto-calculate the number of characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the number of characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b

	Attribute	This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the array passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the array passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_CONST_AIL_TEXT			Specifies to call MfuncParamConstAilText(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const string.	c		e	f	g	a		i		j	k	l	m	n	b
	NumOfItems		Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies for Aurora Imaging Library to auto-calculate the number of characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the number of characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the object passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_FILENAME			Specifies to call MfuncParamFilename(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a file name.	c		e	f	g	a		i		j	k	l	m	n	b
	NumOfItems		Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies for Aurora Imaging Library to auto-calculate the number of characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the number of characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b

		M_IN	Specifies that the parameter of the current user-defined Aurora Imaging Library function will be registered for file reading only.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the parameter of the current user-defined Aurora Imaging Library function will be registered for file writing only.	c		e	f	g	a		i		j	k	l	m	n	b

The following values specify to call type-specific variations of **MfuncParam**() that register pointer parameters, including arrays and const arrays of pointers.

Unless otherwise specified, when specifying values from this table, you must set the AilObjectType parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For registering a pointer type parameter in an Aurora Imaging Library user-defined function																			
ParamType			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	NumOfItems																		
	Attribute																		
M_PARAM_TYPE_CONST_DATA_PTR			Specifies to call MfuncParamConstDataPointer(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const data pointer.																
	NumOfItems		Specifies the number of bytes in the pointer.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the object passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_DATA_PTR			Specifies to call MfuncParamDataPointer(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a data pointer.	c		e	f	g	a		i		j	k	l	m	n	b
	NumOfItems		Specifies the number of bytes in the pointer.	c		e	f	g	a		i		j	k	l	m	n	b

	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the object passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the object passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b

The following values specify to call type-specific variations of **MfuncParam**() that register parameters that handle an *AIL_ID*, including arrays and const arrays of *AIL_ID* s.

When registering *AIL_ID* parameters, you must specify a value for AilObjectType.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For registering an Aurora Imaging Library identifier type parameter in an Aurora Imaging Library user-defined function																		
ParamType			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	NumOfItems																	
	Attribute																	
M_PARAM_TYPE_AIL_ID +			Specifies to call MfuncParamAilId(), which then registers a type *AIL_ID* parameter of the current user-defined Aurora Imaging Library function.															
	NumOfItems		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the object referenced by the *AIL_ID* passed to ParamValuePtr is used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the object referenced by the *AIL_ID* passed to ParamValuePtr is used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b

M_PARAM_TYPE_ARRAY_AIL_ID +			Specifies to call MfuncParamArrayAilId(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles an array of type *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
	NumOfItems		Specify the number of elements in the array.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		This parameter should be set to one or both (M_IN + M_OUT) of the following values.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the objects referenced in the array passed to ParamValuePtr are used for data input.	c		e	f	g	a		i		j	k	l	m	n	b
		M_OUT	Specifies that the objects referenced by the *AIL_ID* passed to ParamValuePtr are used to store new data from the user-defined function.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAM_TYPE_CONST_ARRAY_AIL_ID +			Specifies to call MfuncParamConstArrayAilId(), which then registers a parameter of the current user-defined Aurora Imaging Library function that handles a const array of type *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
	NumOfItems		Specify the number of elements in the array.	c		e	f	g	a		i		j	k	l	m	n	b
	Attribute		Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IN	Specifies that the objects referenced in the array passed to ParamValuePtr are used for data input.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For registering an Aurora Imaging Library identifier type parameter in an Aurora Imaging Library user-defined function.

You can add one or more of the following values to the above-mentioned values to specify the optional settings for an Aurora Imaging Library identifier type parameter.

When you register an Aurora Imaging Library identifier type of parameter, the user-defined function will throw an error if a value other than an *AIL_ID* is passed to it. There are, however, non-standard identifier values that can be accepted. The values specified below specify which non-standard identifier values are valid

in addition to an *AIL_ID*.

There is also the optional of specifying that an Aurora Imaging Library identifier will not be internally modified or compensated during the execution of the user-defined function.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying optional settings for an Aurora Imaging Library identifier type parameter																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALLOW_DEFAULT_ID	Specifies that this parameter will accept **M_DEFAULT** as a valid *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALLOW_NULL_ID	Specifies that this parameter will accept **M_NULL** as a valid *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
M_AS_VALUE	Specifies that this parameter will accept any valid *AIL_ID*, but will not internally modify it. This means different things depending on if this value is added to M_IN or M_OUT. For an Aurora Imaging Library identifier type parameter set to M_IN, the buffer specified by the user is used as the source of information for the user-defined function. When that source buffer is on a different system, the Aurora Imaging Library identifier for that buffer is internally modified if the content of the buffer needs to be copied to another system for processing. This modification is called compensation. If your application uses the Aurora Imaging Library identifiers exactly as specified by the user, your application could crash by inadvertently pointing to a buffer whose content has been copied to another buffer with a compensated Aurora Imaging Library identifier. You can avoid this by adding M_AS_VALUE to M_IN when registering your parameter. This has the effect of keeping the specific value of the Aurora Imaging Library identifier specified by the user, without compensating it. For an Aurora Imaging Library identifier type parameter set to M_OUT, the buffer specified by the user is used as the destination of information for the user-defined function. When you register this parameter without M_AS_VALUE, the user must specify the *AIL_ID* of a previously allocated buffer. When the user-defined function is called, the content of that destination buffer will be filled with new data. When you register this parameter as M_OUT + M_AS_VALUE, the user must specify a pointer. When the user-defined function is called, the pointer will be filled with an *AIL_ID*. The *AIL_ID* could be, for example, a newly-allocated buffer or the result of an inquire.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For registering an Aurora Imaging Library identifier type parameter in an Aurora Imaging Library user-defined function.

You can add one or more of the following values to the above-mentioned values to specify the image buffer type, if AilObjectType is set to M_IMAGE.

If you specify an M_IMAGE object (from the ParamValuePtr parameter) that does not correspond to any of the type(s) selected from the values listed below, an error is returned. If none of the values specified below are added, any M_IMAGE object will be accepted. Note that you cannot specify any of the following if the AilObjectType parameter is set to a combination of values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of image buffer when the AilObjectType is set to M_IMAGE																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DISP	Specifies that the buffer can be displayed.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB	Specifies that the buffer can have data grabbed into it.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC	Specifies that the buffer can be processed.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function

```
void MfuncParamArrayAilDouble (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_DOUBLE *ParamValuePtr,
AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamArrayAilId (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_ID *ParamValuePtr, AIL_INT NumOfItems, AIL_INT64 AilObjectType, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

AilObjectType

See AilObjectType of the main function for a description.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamArrayAilInt (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT *ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamArrayAilInt32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT32 *ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamArrayAilInt64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT64 *ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamArrayAilUint (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT *ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamArrayAilUint32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT32 *ParamValuePtr,
AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamArrayAilUint64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT64 *ParamValuePtr,
AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstArrayAilDouble (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_DOUBLE
*ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstArrayAilId (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_ID *ParamValuePtr,
AIL_INT NumOfItems, AIL_INT64 AilObjectType, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

AilObjectType

See AilObjectType of the main function for a description.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstArrayAilInt (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_INT *ParamValuePtr,
AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstArrayAilInt32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_INT32
*ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstArrayAilInt64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_INT64
*ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstArrayAilUint (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_UINT
*ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstArrayAilUint32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_UINT32
*ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstArrayAilUint64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_UINT64
*ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

Specifies the value of the type *AIL_UINT64* parameter.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstDataPointer (AIL_ID ContextFuncId, AIL_INT ParamIndex, const void *ParamValuePtr,
AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the number of elements in the array.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamConstAilText (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_CONST_TEXT_PTR ParamValuePtr,
AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the length of the string in characters, including the null terminator.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamDataPointer (AIL_ID ContextFuncId, AIL_INT ParamIndex, void *ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the size of the data, in bytes, pointed to by ParamValuePtr.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamFilename (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_CONST_TEXT_PTR ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the length of the string in characters, including the null terminator.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamAilDouble (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_DOUBLE ParamValue)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValue

Specifies the value of the type *AIL_DOUBLE* parameter.

```
void MfuncParamAilId (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_ID ParamValue, AIL_INT64
AilObjectType, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValue

Specifies the value of the type *AIL_ID* parameter.

AilObjectType

See AilObjectType of the main function for a description.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamAilInt (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT ParamValue)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValue

Specifies the value of the type *AIL_INT* parameter.

```
void MfuncParamAilInt32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT32 ParamValue)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValue

Specifies the value of the type *AIL_INT32* parameter.

```
void MfuncParamAilInt64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT64 ParamValue)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValue

Specifies the value of the type *AIL_INT64* parameter.

```
void MfuncParamAilText (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_TEXT_PTR ParamValuePtr, AIL_INT NumOfItems, AIL_INT Attribute)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

NumOfItems

See NumOfItems of the main function for a description.

Specifies the length of the string in characters, including the null terminator.

Attribute

See Attribute of the main function for a description.

```
void MfuncParamAilUint (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT ParamValue)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValue

Specifies the value of the type *AIL_UINT* parameter.

```
void MfuncParamAilUint32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT32 ParamValue)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValue

Specifies the value of the type *AIL_UINT32* parameter.

```
void MfuncParamAilUint64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT64 ParamValue)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValue

Specifies the value of the type *AIL_UINT64* parameter.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncParamCheck

Synopsis

Verify whether parameter checking is required.

Syntax

```
AIL_INT MfuncParamCheck(
    AIL_ID ContextFuncId //in
)
```

Description

This function allows you to verify, from within your user-defined Aurora Imaging Library function, whether parameter checking is enabled or disabled. To enable or disable the checking of parameters, use the MappControl() M_PARAMETER control type.

Call the **MfuncParamCheck**() function prior to checking the parameters of the specified user-defined Aurora Imaging Library function. The return value of the **MfuncParamCheck**() function should dictate whether to execute the functions that perform the parameter checking, or not. Then, to save the parameter checking time for a time-critical user-defined Aurora Imaging Library function, it is sufficient to disable parameter checking using MappControl() with M_CHECK_DISABLE.

Parameter

ContextFuncId

Specifies the identifier of the user-defined Aurora Imaging Library function.

Return value

The returned value is **M_NULL** if no parameter checking is required; otherwise, checking is required.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MfuncParamValue

Synopsis

Read the value of the specified Aurora Imaging Library function parameter.

Syntax

```
void MfuncParamValue(
    AIL_ID ContextFuncId, //in
    AIL_INT ParamIndex,   //in
    void *ParamValuePtr   //out
)
```

Description

This function allows you to read the value of a parameter registered in the caller function of your user-defined Aurora Imaging Library function, from the callee function. The requested value is written to the address specified by the ParamValuePtr parameter.

Parameters

ContextFuncId

Specifies the identifier of the user-defined Aurora Imaging Library function.

ParamIndex

Specifies the index of the parameter within the user-defined Aurora Imaging Library function's parameter list. The index of the first parameter is considered to be one.

ParamValuePtr

Specifies the address of the variable in which to write the value of the specified parameter. The type of the variable depends on the type of the parameter registered in the caller function.

Return value

Returns the value of the specified parameter.

Type-specific versions of the function

```
void MfuncParamValueArrayAilDouble (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_DOUBLE **ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueArrayAilId (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_ID **ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueArrayAilInt (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT **ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueArrayAilInt32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT32 **ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueArrayAilInt64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT64 **ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueArrayAilUint (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT **ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueArrayAilUint32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT32 **ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueArrayAilUint64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT64 **ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueConstArrayAilDouble (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_DOUBLE
**ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueConstArrayAilId (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_ID
**ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueConstArrayAilInt (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_INT
**ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueConstArrayAilInt32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_INT32
**ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueConstArrayAilInt64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_INT64
**ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueConstArrayAilUint (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_UINT
**ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueConstArrayAilUint32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_UINT32
**ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueConstArrayAilUint64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, const AIL_UINT64
**ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueConstAilText (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_CONST_TEXT_PTR
*ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueFilename (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_CONST_TEXT_PTR
*ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueAilDouble (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_DOUBLE *ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueAilId (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_ID *ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueAilInt (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT *ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueAilInt32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT32 *ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueAilInt64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_INT64 *ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueAilText (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_TEXT_PTR *ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueAilUint (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT *ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueAilUint32 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT32 *ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

```
void MfuncParamValueAilUint64 (AIL_ID ContextFuncId, AIL_INT ParamIndex, AIL_UINT64 *ParamValuePtr)
```

Details

Parameters

ContextFuncId

See ContextFuncId of the main function for a description.

ParamIndex

See ParamIndex of the main function for a description.

ParamValuePtr

See ParamValuePtr of the main function for a description.

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib.
DLL	Requires ail11.dll.